

Sumário

1	Introdução	4
2	Artigo 1: Aprendizado de Máquina Supervisionado para Séries Temporais na Área da Saúde (Resumo)	4
2.1	Séries temporais (S)	4
2.2	Aprendizado de máquina supervisionado	4
2.3	Domínio de frequência	4
2.3.1	Transformada de Fourier	5
2.4	Pré-processamento e extração de características	5
2.4.1	Remoção de tendência	5
2.4.2	Interpolação	7
2.4.3	Filtragem	7
2.4.4	Extração de características em séries temporais	8
2.5	Algoritmos e técnicas de classificação e regressão intrínseca (aprendizado de máquina)	8
2.5.1	Algoritmos baseados em distância	8
2.5.1.1	Árvores de decisão	9
2.5.2	Algoritmos baseados em intervalos	9
2.5.3	Algoritmos baseados em dicionários	10
2.5.4	Deep Learning	11
2.5.5	Algoritmos baseados em comitês	12
2.6	Exemplos de classificação	12
2.6.1	Aplicação do algoritmo baseado em distância do vizinho mais próximo (k-NN) com biblioteca aeon	12
2.6.2	Classificação com o InceptionTime	13
2.7	Exemplos de regressão extrínseca	14
2.7.1	Random Convolutional Kernel Transform (ROCKET)	14
2.8	Conclusão	15
3	Artigo 2: A Comprehensive Review of Biosignal Foundation Models	15
3.1	Introdução	15
3.1.0.1	Modelos de machine learning	15
3.1.0.2	Modelos de Deep Learning	16
3.1.0.3	Modelos de Fundação	16
3.2	Biosinais	16
3.2.1	Electroencephalography (EEG)	17
3.2.2	Electromyography (EMG)	17
3.2.3	Electrooculography (EOG)	18
3.2.4	Electrocardiography (ECG)	18
3.2.5	Photoplethysmography (PPG) - Oxímetro	18
3.3	Modelos de fundação	18
3.3.1	Definição de modelos de fundação	18
3.3.1.1	Aspectos chave do espaço latente em AI	19
3.3.1.2	Exemplo Matemático Simples	20

3.3.1.3	Na equação do modelo de fundação	21
3.3.2	Arquiteturas de modelos	21
3.3.2.1	Transformers	21
3.3.2.2	Adendo 1: Treinamento não é a camada do Attention!	25
3.3.2.3	Adendo 2: Exemplo prático do Attention	25
3.3.2.4	Modelos State-Space	26
3.3.2.5	Arquitetura Mamba (State-Space Modelling)	29
3.3.2.6	Conclusão sobre Transformers e SSMS	29
3.3.3	Objetivos de Treinamento Self-Supervised	30
3.3.3.1	Masked Autoencoding	30
3.3.3.2	Autoregressive Models	30
3.3.3.3	Contrastive Learning Methods	31
3.3.4	Métodos de avaliação	32
3.4	EEG Foundation Models	32
3.4.1	Dados EEG e Features	32
3.4.2	Modelos de arquiteturas	32
3.4.2.1	Encoding preliminar ou Convoluções	33
3.4.2.2	Vector Quantization	33
3.4.3	Métodos de avaliação	34
3.5	Modelos de fundação ECG	35
3.5.1	Arquitetura do modelo	35
3.5.2	Considerando relatórios textuais	35
3.5.3	Objetivos de pré-treino	35
3.5.4	Métodos de avaliação	35
3.5.4.1	Como funciona na prática	35
3.6	Outros modelos de fundação unimodais de biosinais	36
3.6.1	Foundations Models para PPG	36
3.6.1.1	Falha matemática no GPT-PPG	36
3.6.2	Foundation Models Para MEG	37
3.6.3	Foundation Models de Biosinais Multimodais	37
3.6.3.1	Early Fusion	37
3.6.3.2	Deep Fusion	38
3.6.3.3	Late Fusion (Contrastive Learning)	39
3.6.4	Métodos de Avaliação	40
3.7	Considerações de Design	40
3.7.1	Seleção de Estratégia de Tokenization	40
3.7.1.1	1. Tokenização Baseada em Pontos (Point-based)	40
3.7.1.2	2. Tokenização Baseada em Patches (Patch-based)	40
3.7.1.3	3. Tokenização Guiada pelo Domínio (Motif-based)	40
3.7.2	Seleção de Arquitetura	41
3.7.2.1	1. Transformers vs. Mamba	41
3.7.2.2	2. Qual estilo de Transformer usar?	41
3.7.2.3	3. O Desafio do Espaço (Multi-canal)	41
3.7.3	Seleção de objetivo de pré-treino	42
3.7.3.1	1. Alinhamento Contrastivo (estilo CLIP)	42

3.7.3.2	2. Objetivos MAE (Masked Autoencoder)	42
3.7.3.3	3. Pré-treinamento Autorregressivo	42
3.7.3.4	4. SimCLR (Contrastive Learning Flexível)	42
3.8	Desafios e direções futuras	42
3.8.1	Comparações justas e benchmarking	42
3.8.2	Explicabilidade e interpretabilidade	43
3.8.2.1	1. Estudos de Ablação (Ablation Studies)	43
3.8.2.2	2. Mapas de Atenção (Attention Maps)	43
3.8.2.3	3. Mapas de Saliência (Saliency Maps)	44
3.8.2.4	4. Testes Baseados em Perturbação (Perturbation-based Tests)	44
3.8.2.5	1. O Perigo do Pré-processamento “Cego”	44
3.8.2.6	2. A Solução: Multiescala e Multirresolução	44
3.8.2.7	3. Fidelidade de Reconstrução como Métrica (MAE)	45
3.8.3	Pronto para aplicação no mundo real?	45
3.8.3.1	1. Eficiência e Recursos Limitados	45
3.8.3.2	2. Privacidade e Ética (Federated Learning)	45
3.8.3.3	3. Regulação Médica	45
3.8.3.4	4. O Desafio da Variabilidade (Generalização)	45
4	Artigo 3: Foundation Models for Physiological Signals	46
4.1	Contexto e Modalidades de Sinais	46
4.2	Adaptação de Modelos de Fundação (TSFMs)	46
4.3	Paradigmas de Pré-treinamento	46
4.4	Taxonomia dos Modelos Atuais	47
4.4.1	Modelos Multimodais de Larga Escala	47
4.4.2	Modelos Focados em Domínio ou Aplicação	47
4.5	Desafios e Futuro	47

Introdução

Estas notas contém anotações sobre artigos iniciais que li para entender os assuntos pertinentes à IC.

Artigo 1: Aprendizado de Máquina Supervisionado para Séries Temporais na Área da Saúde (Resumo)

2.1 Séries temporais (S)

Conjunto de observações (leituras do sensor) feitas em função do tempo. Pode ser unidimensional/univariada ou multidimensional/multivariada.

$$S = (s_1, s_2, \dots, s_n), s_t \in \mathbb{R}^d$$

Componentes:

- Tendência: Direção dos dados ao longo do tempo (ascensão, diminuição, estabilidade)
- Sazonalidade: Flutuações regulares em intervalos fixos (vendas altas no natal)
- Ciclo: Padrões de variação em intervalos irregulares
- Ruído: Variações irregulares/aleatórias

$$S = S_{\text{tend}} + S_{\text{sazon}} + S_{\text{irreg}}$$

Decompor a série temporal e tratar os componentes melhora a qualidade da análise.

2.2 Aprendizado de máquina supervisionado

Aprendizado de máquina é aprender padrões com base em dados. O supervisionado consiste que cada dado possua um atributo alvo ou variável dependente a ser predito. O treinamento é feito a partir de um conjunto de dados rotulados, com o AM induzindo um modelo.

Atributo discreto = rótulo ou classe. Tarefa de classificação é aquela cujo objetivo é obter um modelo que atribua corretamente um rótulo a dados novos.

Caso o conjunto de valores que podem ser conferidos ao atributo alvo seja contínuo (número), a tarefa de se estimar um valor é chamada regressão (como estimativa de calorias gastas durante atividade).

2.3 Domínio de frequência

Podemos representar série temporal por meio de suas componentes de frequência. Pode-se usar a transformada rápida de Fourier, e como resultado cada componente é caracterizada por sua frequência e amplitude (quanto maior a amplitude, maior a influência na série).

A transformada de Fourier pressupõe que a série temporal seja estacionária e linear. Caso não seja, pode ser necessária análise no domínio de tempo-frequência, por meio de Transformada de Fourier de Tempo Curto e suas variantes, ou análise espectral de tempo-frequência. 💡

2.3.1 Transformada de Fourier

Ferramenta matemática que quebra sinal complexo em várias ondas simples (senos e cossenos). Na verdade, os picos de centro de massa da função (Transformada de Fourier) são as frequências das ondas originais que compõem a onda complexa.

Transformada de Fourier - Explicação

Quando uma frequência bate com uma onda originária, vemos um pico no deslocamento do centro de massa da onda. Assim, o gráfico da transformada é $x_{cm}(f)$.

Para amostragem estatística, a transformada rápida de Fourier se reduz a uma somatória (não há função, apenas dados)

$$\sum_{n=0}^{N-1} x_n \cdot e^{-\frac{2\pi k t}{N}}$$

Onde:

- k = Frequência atual (mapeiam-se todas as frequências)
- t = Tempo (índice do array)
- N = Número total de amostras (compensa o crescimento artificial de trocar integral por somatória).

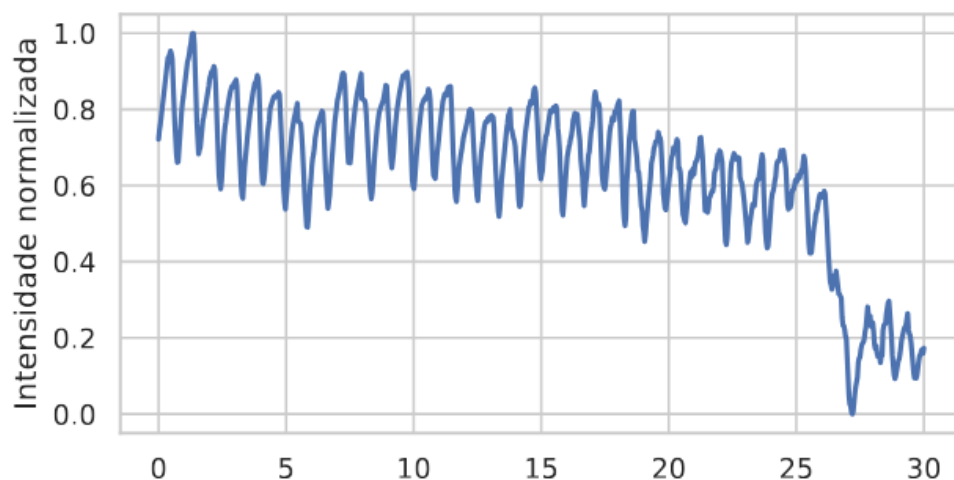
Disciplinas para aprofundar em Fourier: SCC0251, 7600067

A explicação matemática para a definição da Transformada de Fourier está presente em um PDF separado neste mesmo repositório.

2.4 Pré-processamento e extração de características

2.4.1 Remoção de tendência

Fotopletiografia (oxímetro): sensível a pequenas variações de luz externa, causando tendência global decrescente e artefato (queda brusca) ao final.

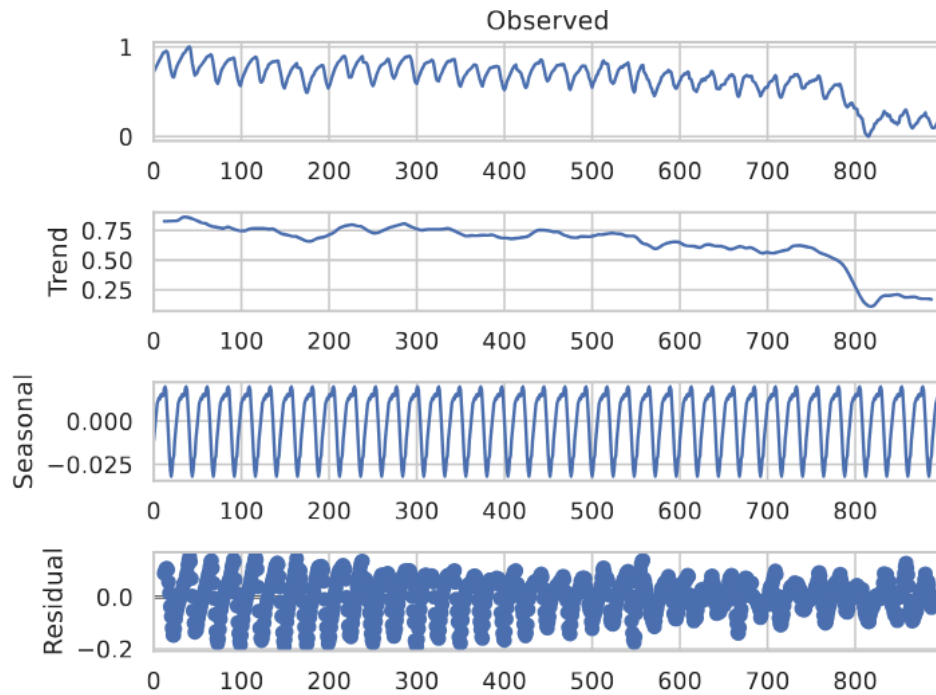


Podemos separar a série temporal em componentes com numpy, statsmodel

```
from statsmodels.tsa.seasonal import seasonal_decompose
import matplotlib.pyplot as plt
```

```
import numpy as np

== Gerar
series = np.genfromtxt('data.tsv', delimiter='\t')
== Decompor
result = seasonal_decompose(series,model='additive',period=24)
== Plotar
plt.figure(figsize=[6,9])
result.plot()
plt.show()
```



Removendo a tendência:

```
import seaborn as sns
sns.set_theme(style='whitegrid')

plt.figure(figsize=[6,3])
== Mostramos apenas sazonal + residual (sem tendência)
sns.lineplot((result.seasonal + result.resid), linewidth=2)
plt.plot()
```

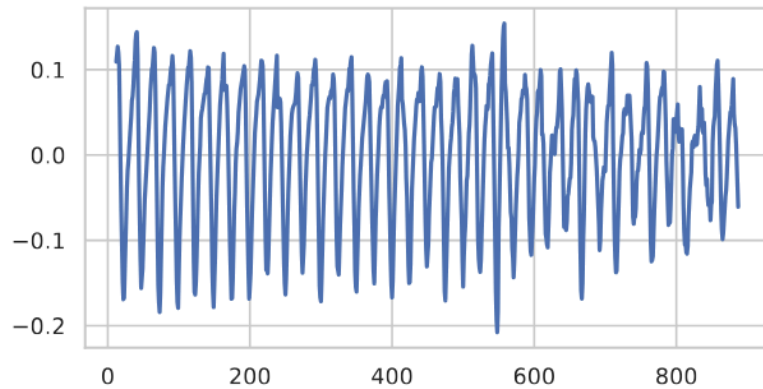
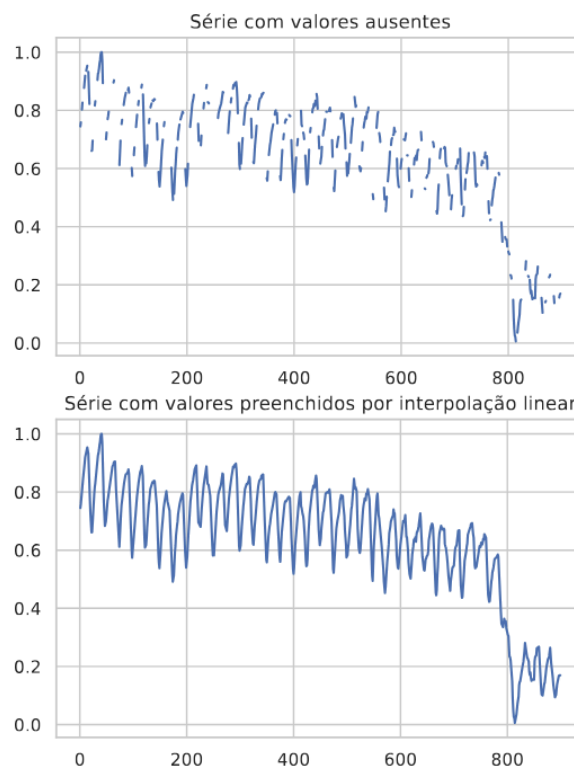


Figura 2.8. Resultado da remoção da componente de tendência.

2.4.2 Interpolação

Para amostragem irregular e valores ausentes. Complementamos valores faltantes com interpolação linear, polinomial, spline (polinômios por parte). Usa-se pandas.



2.4.3 Filtragem

Removem ruídos e imprecisões (culpa do hardware).

Filtro passa-banda: Elimina certa faixa de frequências (imagino que por decomposição por Fourier, elimina frequências e aplica Fourier pra recompor)

Médias móveis: Substituir o valor de um ponto pelo valor médio de vizinhos a ele. Suaviza funções de curto prazo e remove ruídos de alta frequência.

2.4.4 Extração de características em séries temporais

São atributos como máximo, mínimo, número de picos, média, mediana, etc.

tsfel, tsfresh, hctsa, catch22 (apenas 22 atributos canônicos e discriminantes)

Quanto maior o número de picos e vales, maior a complexidade do sinal

2.5 Algoritmos e técnicas de classificação e regressão intrínseca (aprendizado de máquina)

2.5.1 Algoritmos baseados em distância

Operam comparando a similaridade entre as instâncias por meio de distância. A comparação pode ser uma Série contra “Base de Dados” (classificação):

1. Algoritmo pega série atual
2. Compara a distância dela contra uma série padrão
3. Se a distância for pequena, confirma a classificação

Usar distância euclidiana pode falhar em caso de shift no eixo do tempo da série. Assim, usam-se algoritmos que consideram pequenas distorções no eixo do tempo:

- Dynamic Time Warping (DTW)
- Derivative Dynamic Time Warping (DDTW)
- Longest Common Subsequence (LCSS)

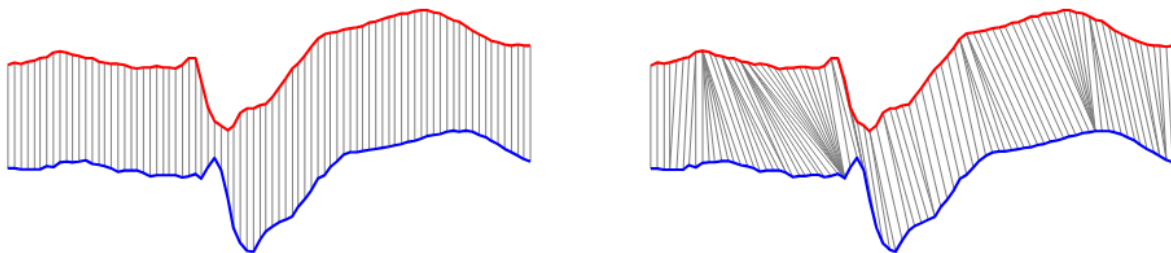


Figura 2.13. Alinhamento de pontos de séries temporais na distância euclidiana (esquerda) e na DTW (direita) [Silva 2017].

Para construir um modelo que não use medidas de distância/similaridade, pode-se usar o algoritmo **k-NN (k-Nearest Neighbors)**. Ele funciona sob a premissa de que “instâncias com características semelhantes tendem a ter resultados semelhantes”;

Ao receber uma nova instância (um dado sem rótulo), o algoritmo:

1. Calcula a **distância** (usando DTW ou Euclidiana, como vimos) entre esse novo dado e todos os outros já rotulados. (k-NN é o motor de decisão, DTW é a régua)
2. Seleciona os **k vizinhos** mais próximos (os que tiverem a menor distância).
3. Define o resultado com base na tarefa:

1. Classificação (Categorias)

O k-NN olha para os k vizinhos e faz uma votação majoritária.

Exemplo: Se $k=3$ e, dos 3 vizinhos mais próximos, 2 são “Falha” e 1 é “Normal”, a nova instância será classificada como “Falha”.

2. Regressão (Números contínuos)

O k-NN olha para os k vizinhos e calcula a média (ou mediana) dos seus valores.

Exemplo: Se você quer prever a temperatura de uma máquina e os 3 vizinhos mais similares registraram 40°C, 42°C e 44°C, a previsão para a nova instância será a média: 42°C.

Outros modelos: Elastic Ensemble, Proximity Forest (constrói árvores de decisão cujo critério é a distância), Proximity Forest 2. 💡

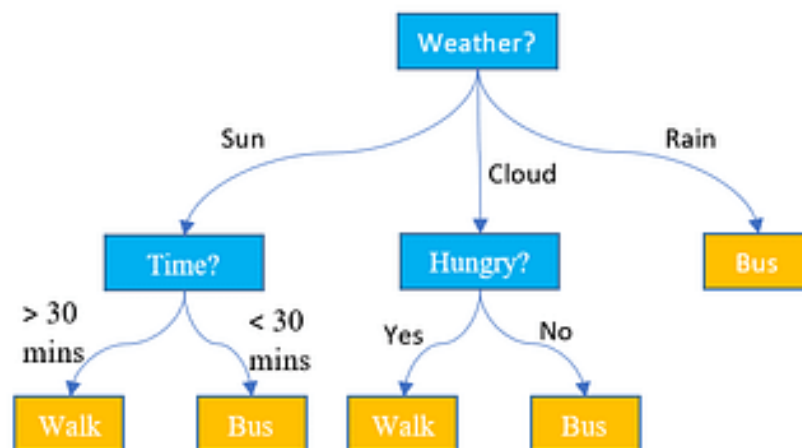
2.5.1.1 Árvores de decisão

Algoritmo de aprendizado de máquina utilizado para resolver problemas de classificação (prever categoria) ou regressão (prever valor numérico).

Na raiz: Ponto de partida, com atributo mais importante para dividir os dados

Nós internos (decisão): Perguntas/condições sobre características dos dados

Ramos: Respostas possíveis (sim/não, maior/menor) que conectam os nós



O algoritmo usa critérios estatísticos para melhor dividir os dados (Entropia ou Gini) e divide até atingir grupos homogêneos.

2.5.2 Algoritmos baseados em intervalos

Extraí características de intervalos ao invés da totalidade. Em geral, esses algoritmos constroem combinações de modelos nas quais cada modelo se ajusta a um ou mais intervalos de um conjunto de séries.

Time Series Forest (TSF): Constrói árvore de decisão para cada intervalo aleatório, com a média, desvio padrão e slope de cada intervalo sendo usados como características de entrada de cada árvore.

Random Interval Spectral Ensemble (RISE): Parecido com TSF, mas as características extraídas de cada intervalo vêm do domínio da frequência e não tempo, como Power Spectrum (PS) e a Autocorrelation Function (ACF).

Canonical Interval Forest (CIF): Igual TSF, mas adiciona às características extraídas os 22 atributos de catch22 (poder discriminatório).

Diverse Representation Canonical Interval Forest (DRCIF): Considera para cada instância sua primeira derivada e o seu periodograma, além dela própria (instância) como entradas para a extração.

2.5.3 Algoritmos baseados em dicionários

Tentam discretizar (dados contínuos, números, em categorias/intervalos) as séries em sequências de padrões/palavras. Criam dicionários que representam a frequência observada de cada palavra nessas séries.

Discriminam ao comparar os dicionários gerados por cada série.

Bag of SFA Symbols (BOSS)

1. Extrai trechos de tamanho w que se movem uma observação à frente a cada iteração (janelas deslizantes)
2. Cada janela passa por Symbolic Fourier Approximation (SFA), Onde seus l primeiros coeficientes espectrais (obtidos por DFT) são discretizados em c compartimentos (ou letras), cada um com o mesmo número de instâncias.
3. Após SFA de cada janela, cada série se torna uma sequência de l letras em um alfabeto de tamanho c .
4. Para palavras iguais consecutivas, considera-se só a primeira (reduz redundância)
5. O n° de ocorrências de cada palavra é contado em sequência, obtendo um conjunto de histogramas, dicionários ou bag of words.
6. Classificação de instâncias feita por algoritmo de 1-NN, com os dicionários obtidos.

Medida de distância do algoritmo (note que apenas as palavras não ausentas na primeira sacola são consideradas, tornando a distância não comutativa):

$$\text{dist}(B_1, B_2) = \sum_{w \in B_1 | B_1(w) > 0} [B_1(w) - B_2(w)]^2$$

2.5.5 Algoritmos baseados em comitês

Não há algoritmo melhor que todos em tudo, cada um se destaca em uma tarefa. Assim, podem-se criar modelos coletivos: distribui ao que possui o melhor desempenho.

Collective of Transformation-Based Ensembles (COTE): Combina modelos a partir de domínios de séries temporais (tempo, frequência, etc)

Hierarchical Vote Collective of Transformation-Based Ensembles (HIVE-COTE): Melhorou COTE.

2.6 Exemplos de classificação

Importante: há um compromisso entre custo computacional e eficácia de cada algoritmo.

2.6.1 Aplicação do algoritmo baseado em distância do vizinho mais próximo (k-NN) com biblioteca aeon

Nesse algoritmo, quando se realiza uma predição, cada instância não rotulada tem sua similaridade calculada com base nas instâncias de treino. Baseando-se nas distâncias para um número de vizinhos k, o algoritmo vai determinar a qual rótulo aquela instância de teste pertence

```
from aeon.classification.distance_based import
KNeighborsTimeSeriesClassifier
import numpy as np

== Extrai dados de treino e teste com numpy
ecg5000_train = np.genfromtxt('ECG5000_TRAIN.tsv',delimiter='\t')
ecg5000_test = np.genfromtxt('ECG5000_TEST.tsv',delimiter='\t')
== Define eixos
x_train, y_train = ecg5000_train[1:], ecg5000_train[0]
x_test, y_test = ecg5000_test[1:], ecg5000_test[0]
== Aplica k-NN aos dados de treino
knn = KNeighborsTimeSeriesClassifier()
knn.fit(x_train, y_train)
== usa k-NN pra predizer
y_pred = knn.predict(x_test)

== Determina acurácia
print(accuracy_score(y_test, y_pred))
```

A figura abaixo é a matriz de confusão para o algoritmo KNN No conjunto de dados. É uma tabela usada para medir o desempenho de um modelo de classificação. Ela mostra onde o modelo acertou e, principalmente, com o que ele se **confundiu**.

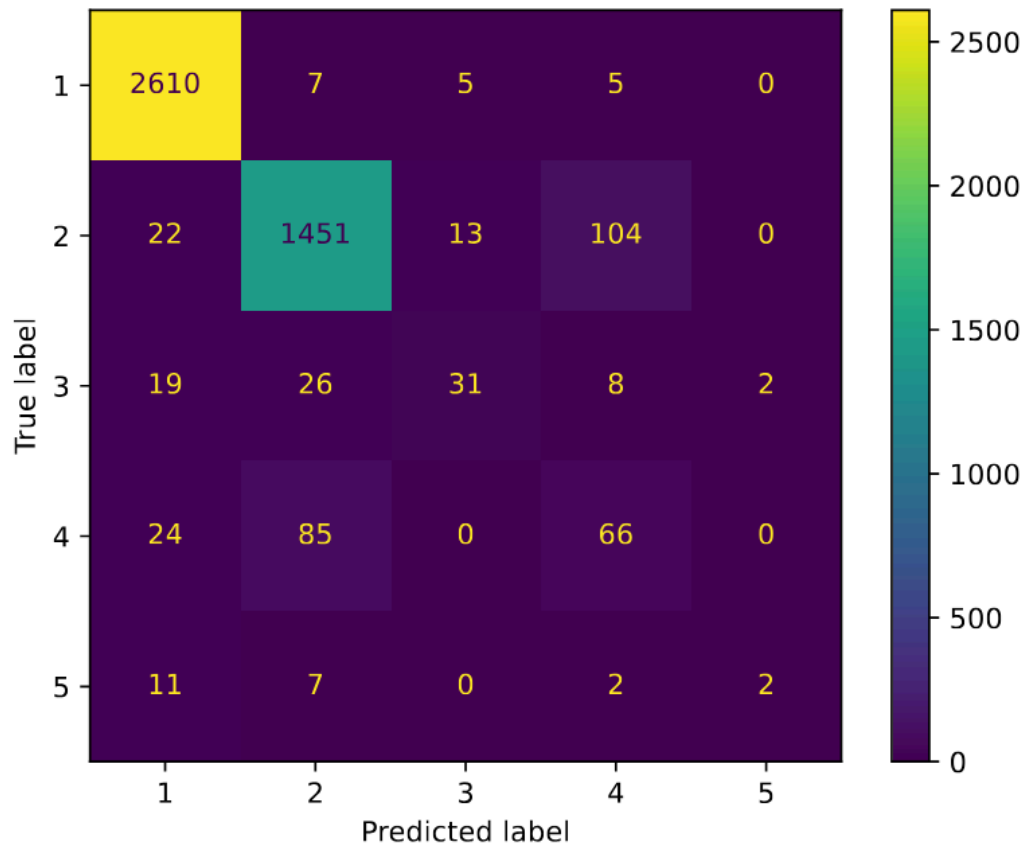


Figura 2.21. Matriz de Confusão gerada pelo Algoritmo KNN.

Como ocorre um desbalanceamento de classes neste conjunto, onde a maioria das classes são pertencentes aos rótulos 1 e 2, as outras classes que possuem poucos exemplos concentram grande parte dos erros do algoritmo

2.6.2 Classificação com o InceptionTime

O InceptionTime baseia-se em InceptionBlocks para capturar padrões e relações lineares e não lineares na estrutura das séries temporais. Contudo para utilizá-lo é preciso transformar as séries que normalmente são carregadas em um array de duas dimensões para três dimensões, sendo elas: o número de instâncias, o número de canais e a última dimensão o tamanho da série ou a quantidade de observações.

```
from tsai.all import *
import numpy as np
```

```
ecg5000_train = np.genfromtxt('ECG5000_TRAIN.tsv', delimiter='\t')
ecg5000_test = np.genfromtxt('ECG5000_TEST.tsv', delimiter='\t')
```

```
x_train, y_train = ecg5000_train[1:], ecg5000_train[0]
x_test, y_test = ecg5000_test[1:], ecg5000_test[0]
```

```
x_train = np.reshape(x_train, (x_train.shape[0], 1, x_train.shape[1]))
x_test = np.reshape(x_test, (x_test.shape[0], 1, x_test.shape[1]))
```

```

train_ds = TSDataset(x_train, y_train - 1, types=(TSTensor, TSLabelTensor))
train_dl = DataLoader(train_ds, bs=128, num_workers=0)

test_ds = TSDataset(x_test, y_test - 1, types=(TSTensor, TSLabelTensor))
test_dl = DataLoader(test_ds, bs=128, num_workers=0)
dls = DataLoaders(train_dl, test_dl, device=default_device())

num_classes = len(np.unique(y_train))
model = InceptionTime(1, num_classes)

learn = Learner(dls, model, loss_func=nn.CrossEntropyLoss(), metrics=accuracy)
learn.fit_one_cycle(25, lr_max=1e-2)

preds, labels = learn.get_preds(dl=test_dl)
print(accuracy_score(preds.argmax(dim=1).numpy(), labels.numpy()))

```

Vendo a matriz de confusão, ele é parecido com o KNN.

2.7 Exemplos de regressão extrínseca

2.7.1 Random Convolutional Kernel Transform (ROCKET)

O ROCKET utiliza um grande número de filtros convolucionais (em geral, 10000), construídos aleatoriamente a partir de diversos parâmetros, aplicando-os às séries temporais e extraindo características da série resultante, por padrão, valor máxima (max) e proporção de valores positivos (ppv). Essas características formam uma grande tabela atributo-valor, que é então usada como entrada para um classificador linear.

Filtros convolucionais

São matrizes de números (pequenas matrizes 2D, geralmente 3x3 ou 5x5) aplicadas a imagens ou dados para extrair características específicas, como bordas, cantos, ou para realizar suavização/desfoque

Abaixo, utiliza-se ROCKET com aeon para a predição de frequência cardíaca.

```

from aeon.datasets import load_from_tsfile
from aeon.transformations.panel.rocket import Rocket

from sklearn.pipeline import Pipeline
from sklearn.linear_model import RidgeCV
from sklearn.metrics import mean_squared_error

== Carregar o dataset para a memoria
X_train, y_train = load_from_tsfile('BIDMCHR_TRAIN.ts')
X_test, y_test = load_from_tsfile('BIDMCHR_TEST.ts')

== Definir modelo
model = Pipeline([
    ('transformer', Rocket()),

```

```

    ('regressor', RidgeCV())
])

== Ajustar modelo aos dados
model.fit(X_train, y_train)

== Obter previsões
y_pred = model.predict(X_test)

print(mean_squared_error(y_test, y_pred, squared=False))

```

2.8 Conclusão

- Algoritmos modernos para aprendizado de máquina em séries temporais enfrentam problema na balança entre eficiência e eficácia. Algoritmos mais precisos são os mais custosos.
- Por ser saúde, necessitam-se de algoritmos mais precisos mas não tão custosos.

Artigo 2: A Comprehensive Review of Biosignal Foundation Models

3.1 Introdução

Electroencephalography (EEG) pode ser usada para diagnosticar epilepsia ou distúrbios do sono, e electrocardiography (ECG) permite detectar prematuramente arritmia cardíaca. Biosinais são agora usados em aplicações emergentes como interfaces cérebro-computador (BCI).

A inspeção visual de biosinais por técnicos não é confiável, pois muita informação relevante não é facilmente visível em uma imagem ou sinal. Vários métodos de processamento de sinais foram usados para extrair informação latente, como análise de Fourier e transformada de Wavelet.

3.1.0.1 Modelos de machine learning

Foram os primeiros a serem usados para analisar biosinais.

- **SVM (Support Vector Machines):** Imagine que você tem pontos de duas cores em uma mesa. O SVM tenta desenhar a linha (ou “fronteira”) perfeita que separa essas cores com a maior distância possível entre elas.
- **Random Forests (Florestas Aleatórias):** É um conjunto de várias “árvores de decisão”. O modelo faz várias perguntas sim/não aos dados e combina as respostas de todas as árvores para chegar a uma conclusão mais precisa.
- **HMM (Hidden Markov Models):** Muito usados para sequências (como fala ou batimentos cardíacos). Eles tentam prever o próximo estado de algo baseando-se em probabilidades de estados que não vemos diretamente.

Não conseguiam entender dados “brutos” sozinhos. Features (características) importantes eram extraídas manualmente.

Ex: Reconhecer maçã - Tem que programar para reconhecer o nível de “vermelho”, “circularidade”, “tamanho médio”. Se ela fosse verde, o modelo já falha.

3.1.0.2 Modelos de Deep Learning

Depois, vieram modelos que criam as features automaticamente.

- **CNN (Redes Neurais Convolucionais):** São especialistas em **espaço**. Elas conseguem olhar para uma imagem ou um sinal e identificar automaticamente padrões geométricos, bordas e texturas. Elas “escaneiam” os dados brutos em busca de relevância.
- **RNN (Redes Neurais Recorrentes) e LSTM:** São especialistas em **tempo/sequência**. Elas possuem uma espécie de “memória” que permite entender que o que aconteceu há um segundo influencia o que está acontecendo agora (essencial para fala, tradução de textos ou sinais de sensores).

Possuem dificuldade com dados em condições diferentes do treinamento, não há generalidade.

Também requerem treinamento supervisionado com dados que podem ser limitados

Como o biosinal é complexo e é resultado de muitas variáveis somadas, quando as variáveis (atividade fisiológica, ambiente, estímulo) apresentam comportamento diferente, a performance do modelo cai.

3.1.0.3 Modelos de Fundação

Em resposta à isso, Modelos de Fundação possuem habilidade de aprender poderosas representações de dados que podem generalizar além das condições dos seus dados de treinamento. Teve sucesso em Computer Vision e LLMs.

3.2 Biosinais

São medidas que capturam a atividade fisiológica de um organismo vivo sobre o tempo. Abaixo: EEG (atividade cerebral), ECG (função cardíaca), EMG (atividade muscular), PPG (mudanças no volume do sangue).

São expressos como séries temporais que refletem interação biológica complexa entre múltiplas escalas temporais e espaciais.

- **Escala temporal:** Um sinal pode ter batidas rápidas (milissegundos) e ciclos lentos (minutos ou horas).
- **Escala de frequência:** No corpo, um músculo contraindo gera frequências diferentes de um coração batendo, e o modelo precisa separar esse “barulho”.
- **Informação espacial:** Se você coloca 20 eletrodos na cabeça de alguém (EEG), a posição de cada um importa. O sensor na testa capta coisas diferentes do sensor na nuca. O computador não precisa saber apenas o que o Sensor A está dizendo, mas também o quão longe o Sensor A está do Sensor B.

Biosinais são difíceis de analisar, diferentemente de imagens ou textos.

- **Baixa Relação Sinal-Ruído (Low SNR):** Biosinais são eletricamente muito fracos, e facilmente apresentam ruídos: Movimento muscular, interferência elétrica, etc.

- **Inconsistência de Hardware e Condições:** os biosinais mudam conforme o equipamento. O hardware de uma marca pode detectar de modo diferente do hardware de outra marca. A posição dos sensores também importa, além da umidade do ambiente, etc.
- **Variabilidade Inter-sujeito (O maior desafio):** O coração de uma pessoa pode ser levemente inclinado para o lado; o crânio de outra pode ser mais espesso. Idade, nível de condicionamento físico e uso de medicamentos mudam a forma das ondas do sinal.

IA na saúde não é como em imagens ou textos. Os biosinais não têm uma “forma” visual óbvia para nós e são instáveis. O que parece uma doença em um sinal pode ser apenas um movimento brusco do paciente.

3.2.1 Electroencephalography (EEG)

O EEG grava pequenas flutuações de tensão geradas pela atividade cerebral dos neurônios no cérebro. Os sinais primeiro se originam dos potenciais dos neurônios piramidais no córtex cerebral.

Um único neurônio disparando é fraco demais para ser detectado fora da cabeça. O EEG só funciona porque milhares de neurônios disparam ao mesmo tempo (sincronia). Essa soma de micro-correntes cria um campo elétrico forte o suficiente para viajar através do tecido cerebral, líquido cefalorraquidiano, crânio e couro cabeludo.

- **fMRI (functional Magnetic Resonance Imaging):** Mede atividade cerebral detectando mudanças na oxigenação do sangue e fluxo em resposta a atividade neural. Resolução temporal baixa (segundos).
- **EEG (Escalpo/Convencional):** Resolução temporal altíssima (milissegundos) em comparação com fMRI, e os eletrodos são colados na pele.
 - **Low SNRT:** Sinal ruidoso e fraco) por causa do crânio.
 - **Resolução temporal alta:** Por ser rápido, pode detectar **crises epilépticas** e mudanças rápidas na **atenção**
 - **Resolução espacial ruim:** Perde em **resolução espacial** para o fMRI. Apenas 21 eletrodos (sensores) para um cérebro complexo é muito pouco, até mesmo 200. Cada local possui um sinal diferente. O fMRI possui precisão de sub-milímetros na imagem.
 - **Big Data:** Como é fácil (não invasivo, pode ser bebê ou atleta) e barato de usar, conseguimos criar **grandes bases de dados** (datasets),
- **iEEG (Intracraniano):** Os eletrodos são colocados cirurgicamente **dentro** da cabeça, encostados no cérebro.
 - **Vantagem:** O sinal é incrivelmente nítido porque não precisa atravessar o osso.
 - **Desvantagem:** Requer cirurgia cerebral (geralmente feita em casos graves de epilepsia).

3.2.2 Electromyography (EMG)

O EMG mede a atividade cerebral gerada pelas fibras musculares por meio de eletrodos colocados na pele ou inseridos dentro do tecido muscular.

Os sinais podem ser usados para detectar distúrbios neuromusculares, ou para controlar próteses decodificando sinais musculares e traduzindo eles em comandos. A forma de onda do EMG é altamente estocástica (aleatória, mas que evolui ao longo do tempo) e irregular, refletindo picos nos potenciais de ação das unidades motoras durante contrações.

Os sinais EMG são muito diferentes dependendo da colocação do sensor e podem variar muito entre indivíduos devido a diferenças anatômicas.

3.2.3 Electrooculography (EOG)

O EOG rastreia o movimento ocular usando eletrodos bipolares que medem a diferença de potencial entre dois pontos próximo ao olho.

O olho atua como um dipolo devido à diferença de potencial entre a retina e a córnea, gerando um sinal elétrico forte quando move ou pisca.

Pode ser usado para medir a atenção visual, detectar estágios de sono como REM, ou controlar interfaces por gestos do olho.

Mas sinais EOG podem ser pegos perto de canais EEG, o que pode tornar difícil de separar da real atividade neural.

3.2.4 Electrocardiography (ECG)

Um dos biosinais mais usados na medicina. Captura o batimento cardíaco ao gravar as ddp's causadas pela despolarização e repolarização do músculo do coração.

3.2.5 Photoplethysmography (PPG) - Oxímetro

Mede o quanto o volume de sangue muda com um pequeno sensor de luz. Rastreia batimento cardíaco, oxigenação do sangue ou nível de stress estimado. Sensível a artefatos de movimento e condições do ambiente.

3.3 Modelos de fundação

É um modelo de larga escala. Diferente de modelos de deep-learning para tarefas específicas, um modelo de fundação foca em capturar representações universais entre domínios ou modalidades, sendo o esqueleto de um modelo de propósito geral.

3.3.1 Definição de modelos de fundação

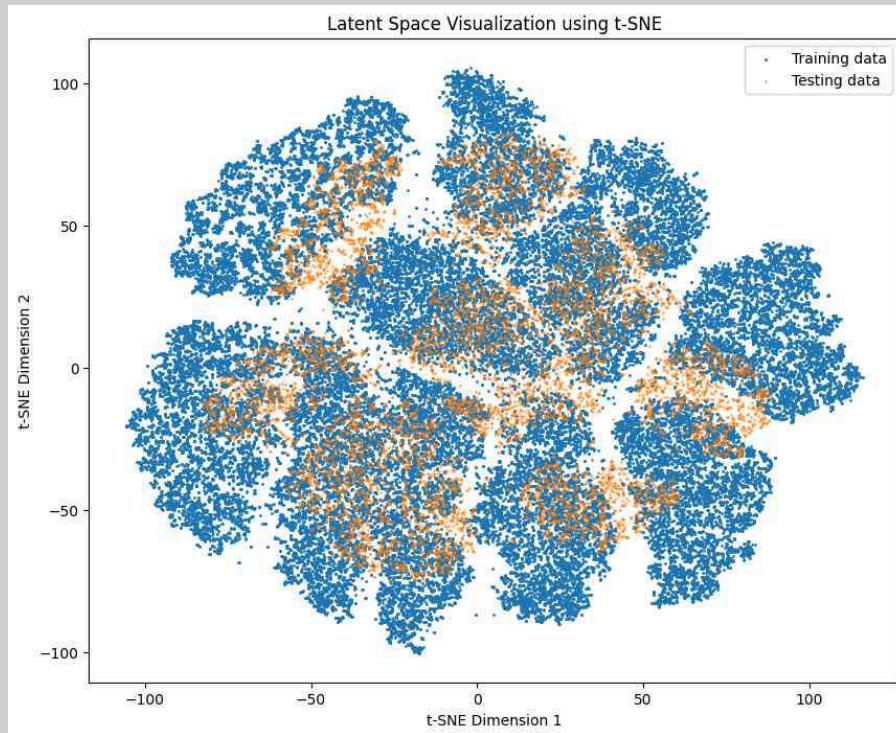
Definimos $D = (x_i, y_i)_{i=1}^N$ como um dataset de treino massivo. x_i = sinais de entrada (de várias modalidades/tarefas), y_i = labels (da tarefa).

Um modelo de fundação aprende uma f_θ que transforma os dados brutos de entrada (X) em um espaço latente compartilhado (Z). θ são os parâmetros treináveis.

$$f_\theta : X \rightarrow Z$$

Espaço latente

Representação matemática comprimida, de menor dimensão e abstrata (vetores) de dados complexos onde o modelo entende o significado e o contexto dos dados.



Permite à IA generativa entender, manipular e criar novo conteúdo.

3.3.1.1 Aspectos chave do espaço latente em AI

1. Compressão de dados

Dados brutos possuem muita informação irrelevante. O modelo comprime inputs altamente dimensionais em apenas features (características) essenciais, reduzindo a complexidade enquanto preserva os dados.

2. Geometria

Itens similares estão próximos no espaço.

- Se o modelo for de linguagem, as palavras “cachorro” e “poodle” estarão geograficamente perto uma da outra.
- “Cachorro” e “Geladeira” estarão muito distantes.

3. Continuidade e navegabilidade

São contínuos, permitindo interpolação (transições suaves entre dois pontos de dados) e manipulação.

4. Desentrelaçamento

Processo de separar fatores latentes de variação subjacentes nos dados (como cor, forma e tamanho em uma imagem) para que cada um possa ser manipulado de modo independente. Melhora a interpretabilidade e representação.

Assim, dá para realizar “contas” com esses conceitos (vetores). Em Processamento de Linguagem Natural (NLP) é:

Rei - Homem + Mulher = Rainha

Isso ocorre porque o modelo isola o vetor de “gênero” e de “realidade” no espaço latente.

O objetivo do treinamento de um modelo de fundação é encontrar os melhores parâmetros (θ^*) que minimizem o erro (perda) durante o aprendizado:

$$\theta^* = \arg \min_{\theta} E_{x \sim D} [L(f_{\theta}(x), t(x))]$$

- **argmin θ (Argumento do Mínimo)**: Não indica o valor mínimo da função em si, mas sim o **valor dos parâmetros (θ)** que faz a função atingir seu menor resultado possível.
- **$E_{x \sim D}$ (Esperança Matemática / Média)**: Representa o **valor esperado** ou a média ponderada sobre todos os dados do conjunto D.
 - **O que faz**: Garante que o modelo aprenda com o conjunto de dados inteiro, e não apenas com um exemplo isolado. O objetivo é minimizar o erro médio em toda a base.
- **L (Função de Perda / Loss Function)**: É a métrica que calcula a “distância” ou o **erro** entre o que o modelo previu ($f_{\theta}(x)$) e o alvo real ($t(x)$).
 - **O que faz**: Funciona como uma nota. Se a perda é alta, o modelo errou muito; se é baixa, o modelo está aprendendo corretamente a tarefa (como prever a próxima palavra).
- **$t(x)$ (Auto-supervisão)**: O modelo não precisa de humanos rotulando tudo. Ele cria seus próprios alvos, como:
 - **Masked prediction**: Adivinhar palavras escondidas em uma frase.
 - **Next-step prediction**: Prever a próxima palavra ou frame.

Uma vez que o modelo aprendeu essa base geral (pré-treinamento), ele pode ser **ajustado (fine-tuning)** para tarefas específicas, como tradução médica ou análise de sentimentos, sem precisar começar do zero.



Função arg min (argumentum minimi)

Enquanto o min busca o **valor mais baixo** de uma função, o argmin busca o **valor da variável** que produz esse resultado mínimo.

3.3.1.2 Exemplo Matemático Simples

Dada a função $f(x) = (x-3)^2 + 10$:

1. O que é o min $f(x)$?

O valor mais baixo que essa função atinge é **10** (quando o parêntese zera).

2. O que é o argmin $x f(x)$?

É o valor de x que faz a função chegar no 10. Neste caso, **3**.

$$\min f(x) = 10$$

$$\operatorname{argmin} f(x) = 3$$

3.3.1.3 Na equação do modelo de fundação

$$\theta^* = \arg \min_{\theta} E_{x \sim D} [\mathcal{L}(f_{\theta}(x), t(x))]$$

- $\mathcal{L}$: É a “montanha” de erro do modelo. compara o que o modelo previu (f_{θ}) com a resposta correta (t).
- $E_{\{x \sim D\}}$: Como o modelo não pode ser bom em apenas uma frase, mas sim em todo o conjunto de dados (D), o E calcula a **média do erro** para todos os exemplos.
- θ : São os pesos (os números a serem ajustados dentro das matrizes aprendíveis W^Q , W^K , W^V).
- θ^* : Representa a configuração exata desses pesos que faz o erro ser o menor possível.

</aside>

3.3.2 Arquiteturas de modelos

Todo modelo de fundação é composto de um esqueleto principal, precedido/sucedido por módulos menores (para filtrar/codificar os dados antes de passar para o backbone, por exemplo). Os backbones mais comuns para modelos de fundação são **Attention-based (Transformers)** e **State-Space-Model (SSM)-based (Mamba)**.

3.3.2.1 Transformers

Transformers, the tech behind LLMs | Deep Learning Chapter 5

Diferente de modelos antigos que processam palavras uma por uma, o Transformer processa todas as palavras (**tokens**) em paralelo. O mecanismo de auto-atenção permite que o modelo decida quais palavras na sequência são mais relevantes entre si, independentemente da distância.

Transformers não é pra treino, é para processar inputs com um modelo treinado!

Dada uma sequência de tokens $X = [x_1, \dots, x_T] \in \mathbb{R}^{\{T \times d\}}$, cada camada do transformer computa representações como:

$$H = \text{Attention}(XQ, XK, XV) = \text{softmax}\left(\frac{XQ(XK)^T}{\sqrt{d_k}}\right)XV$$

O cálculo principal utiliza três matrizes de projeção aprendíveis: **Q** (Query), **K** (Key) e **V** (Value).

- $XQ(XK)^T$: Calcula a afinidade (relevância) entre cada palavra e todas as outras.
- d_k : Dimensionalidade da Key. Um fator de escala para os números não explodirem.
- **softmax**: Normaliza o resultado e transforma as afinidades em probabilidades (pesos de 0 a 1).
- XV : Aplica esses **pesos** aos valores originais para gerar a nova representação (H).

Para cada token x_t , sua representação h_t é atualizada como uma soma ponderada de todos os valores dos tokens, permitindo ao modelo capturar dependências de longo-alcance.

O transformer é construído com blocos de encoder/decoder empilhados, onde cada bloco encoder contém módulos multi-head self-attention e redes feedforward posicionais (Camadas neurais simples que processam cada palavra individualmente após a atenção ter misturado os contextos), e os blocos decoder incorporam a atenção cruzada para os outputs do encoder.

A formulação original adiciona encodings posicionais sinusoidais aos embeddings de entrada para capturar informações sobre a ordem dos tokens na sequência.

!image.png

Enquanto transformers provêm modelagem de contexto global, a complexidade da atenção $\mathcal{O}(T^2)$ limita a escalabilidade para sinais longos. (Se há 1000 palavras, o modelo faz 1.000.000 de comparações na matriz de afinidade)

Encoder e Decoder

O Encoder (O “Entendedor”)

O papel dele é ler a entrada (ex: uma frase em português) e extrair todo o significado e contexto.

- **O que ele faz:** Ele olha para cada palavra, vê como elas se relacionam (usando a Atenção que discutimos) e transforma tudo em um **vetor abstrato de conceitos**.
- **Resultado:** No final do Encoder, você não tem mais palavras, mas sim uma representação numérica densa de “o que essa frase quis dizer”.

O Decoder (O “Gerador”)

O papel dele é pegar esse “mapa de conceitos” gerado pelo Encoder e transformá-lo em uma nova sequência (ex: a tradução para o inglês).

- **O que ele faz:** Ele gera uma palavra por vez. Para decidir a próxima palavra, ele olha para duas coisas:
 1. O que ele já escreveu até agora.
 2. O “mapa de conceitos” que o Encoder enviou.
- **Resultado:** Uma nova frase que mantém o sentido da original.

Hoje, nem todos os modelos usam os dois juntos:

- **Só Encoder (ex: BERT):** Ótimo para classificar textos, analisar sentimentos ou extrair dados. Ele entende, mas não “escreve” bem
- **Só Decoder (ex: GPT/ChatGPT):** É o que chamamos de modelo autorregressivo. Ele é treinado especificamente para prever o próximo token e gerar textos longos e fluidos.
 - ▶ Em modelos como o GPT, não há uma “frase de entrada” separada de uma “frase de saída” (como na tradução). Tudo é tratado como uma única sequência de texto.
 - ▶ O que você escreveu no prompt (a entrada) é tratado pelo Decoder como se fossem as primeiras palavras que ele mesmo gerou (“encoder”)

Arquitetura	O que faz?	Principal Objetivo	Exemplo de Uso
Encoder	Transforma o texto em um “mapa” de números que	Entendimento: Classificar, rotular	BERT (Usado para buscas no Google).

	captura o contexto de todas as palavras.	ou extrair informações.	
Decoder	Gera texto um token por vez, baseando-se no que foi escrito antes.	Geração: Criar textos, diálogos e completar frases.	GPT (ChatGPT, Llama).
Encoder-Decoder	O Encoder entende a entrada e o Decoder “traduz” esse entendimento para outra língua ou formato.	Transformação: Mapear uma sequência complexa em outra.	T5 ou Tradutores (Português → Inglês).

Matrizes Aprendíveis

cada palavra entra no modelo como um vetor estático (um conjunto fixo de números). Se o modelo não pudesse mudar nada, ele seria rígido. As **matrizes aprendíveis** são os “pesos” que o modelo ajusta **durante o treinamento**.

(OBS: O H acima é para um modelo já treinado, ele apenas processa o input. As matrizes aprendíveis não mudam no H)

- **O conceito de “Aprendível”:** São tabelas de números (parâmetros θ) que começam aleatórias. Durante o treino, o algoritmo de **Backpropagation** as ajusta para que o erro (L) diminua.
- **Q,K e V não são as matrizes aprendíveis**, eles são os **resultados** (projeções) da multiplicação. As matrizes aprendíveis (os “pesos”) são geralmente denotadas como W^Q , W^K e W^V . Para cada entrada X (o seu texto transformado em números), o modelo faz três contas separadas:
 1. $Q = XW^Q$ (Cria a **Query**)
 2. $K = XW^K$ (Cria a **Key**)
 3. $V = XW^V$ (Cria o **Value**)
- **As Projeções (Q,K,V):**
 - A **Query (Q)** é um vetor que destaca certas propriedades (ex: “sou um substantivo que aceita adjetivos de estado”).
 - A **Key (K)** é vetor que projeta propriedades de “adjetivo/estado”.
 - **V (Value/Valor):** A informação real que a palavra carrega. Faz o filtro do X para extrair apenas a informação que será útil para a tarefa atual. Com o V, cada “cabeça” de atenção pode ter sua própria matriz W^V . Isso permite que:
 - A **Cabeça 1** transforme X em um V focado em **sintaxe** (gramática).
 - A **Cabeça 2** transforme X em um V focado em **semântica** (significado).

- ▶ Quando **Q** e **K** são multiplicados, se o “quebrado” for um adjetivo que costuma descrever “banco” (baseado nos bilhões de textos que o modelo leu no treino), o resultado numérico é alto. É a pergunta e a resposta!

Resumo do Fluxo

1. A palavra entra como um vetor **X**.
2. **X** é multiplicado pelas matrizes aprendíveis para criar **Q**, **K** e **V**.
3. **Q** e **K** das palavras do input interagem para gerar scores de afinidade.
4. A **Softmax** normaliza esses scores.
5. O resultado da Softmax multiplica **V** para gerar a resposta final daquela camada.

Matriz de afinidade

$$XQ(XK)^T$$

O que estamos fazendo aqui? O **T** é a transposta de **XK**. Na álgebra linear, multiplicar uma matriz por outra transposta é a forma matemática de calcular o **produto escalar** entre todas as combinações de linhas das duas matrizes.

Assim, estamos comparando cada **Query** (pergunta) **Q** com cada **Key** (etiqueta) **K** das outras palavras. Ou seja, comparamos todas as palavras entre si.

Imagine a frase: “**O banco quebrado**”. Temos 3 tokens (**t1**, **t2**, **t3**). Cada um deles gera seu próprio **Q**, **K**, **V** ao se multiplicar pela matriz aprendível.

Para saber o quanto o “banco” deve prestar atenção no “quebrado”, o modelo faz um **produto escalar** entre a **Query do banco** (**Q2**) e a **Key do quebrado** (**K3**).

!image.png

O produto escalar de dois vetores vai medir a similaridade entre eles. Se apontarem para a mesma direção (no contexto de espaço latente, se estiverem próximos), o resultado é um número muito alto. Se forem irrelevantes entre si, teremos um resultado próximo de zero ou negativo.

O Fluxo de Afinidade para o “banco”:

1. Pegamos o vetor Q_{banco} .
2. Multiplicamos por K_O , K_{banco} , K_{quebrado} .
3. Isso nos dá 3 pontuações (ex: 1.2, 10.5, 8.1).
4. Passamos esses 3 números pela **Softmax** (ela só normaliza os valores para que a soma seja igual a 1).
5. O resultado são os pesos (ex: 0.00008, 0.916, 0.083).

Softmax

A **Softmax** é uma função de ativação que converte um vetor de números reais (que podem ser negativos ou muito grandes) em uma **distribuição de probabilidade**.

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

- **Soma 1:** Ela garante que a soma de todos os valores de saída seja exatamente 1 (ou 100%).
- **Amplifica diferenças:** Ela faz com que o maior valor se destaque muito mais do que os outros (por causa da exponencial)
- **Elimina números negativos:** Mesmo que um score de afinidade fosse -5 , o e^{-5} seria um número positivo muito pequeno, mantendo a probabilidade válida.

3.3.2.2 Adendo 1: Treinamento não é a camada do Attention!

O treinamento é o momento em que o modelo olha para bilhões de frases e usa a equação:

$$\theta^* = \arg \min_{\theta} E_{\{x \sim D\}} [L(f_{\theta}(x), t(x))]$$

- Ele começa com as matrizes WQ, WK e WV cheias de números aleatórios.
- Ele erra a previsão, calcula o erro ($\mathcal{L}$) e ajusta os números dessas matrizes.
- Ao final, essas matrizes “aprendem” as regras da linguagem.

3.3.2.3 Adendo 2: Exemplo prático do Attention

Exemplo: “O **banco** estava quebrado.”

Imagine que o modelo já foi treinado com o conjunto $\mathcal{D}$. Durante esse treino, ele aprendeu que a palavra “banco” aparece em dois contextos diferentes:

1. Perto de: **sentar, madeira, praça.**
2. Perto de: **dinheiro, quebrado, juros, depósito.**

Lembre-se: a proximidade é dada pela localização dos vetores das palavras no espaço latente.

1. O Estado Inicial (X)

Quando a palavra “**banco**” entra no modelo, ela é representada por um vetor de números (ex: $[0.5, -0.2, 0.1]$). Esse vetor é genérico e carrega um pouco de “assento” e um pouco de “instituição”.

2. O Mecanismo de Atenção atuando

Na frase “O banco **estava** quebrado”:

- A **Query (Q)** é um vetor que destaca certas propriedades (ex: “sou um substantivo que aceita adjetivos de estado”).
- A **Key (K)** é vetor que projeta propriedades de “adjetivo/estado”.
- O **Value (V)** extrai apenas a informação útil.
- A **Softmax** dá um peso alto para a relação entre “banco” e “quebrado”.

3. O Resultado (H)

O valor de saída H para a palavra “banco” não será mais o $[0.5, -0.2, 0.1]$ original.

Ele será uma **combinação matemática** (os pesos vêm da softmax):

$$H_{\text{banco}} = (\text{Peso}_1 \cdot V_{\text{banco}}) + (\text{Peso}_2 \cdot V_{\text{quebrado}})$$

O vetor H resultante agora “puxou” características do vetor da palavra **“quebrado”**. No espaço matemático do modelo (o espaço latente $\mathcal{Z}$), esse novo vetor de “banco” se moveu para uma região onde residem conceitos de falência ou problemas financeiros, e se afastou da região de “móveis de madeira”.

Assim, a mistura de vetores gera um contexto.

3.3.2.4 Modelos State-Space

Alternativa aos transformers para processar sequências longas. Essa arquitetura modela dinâmicas temporais usando estados latentes de tempo-contínuo governados por um sistema dinâmico linear.

A ideia é tratar o processamento de dados não como uma atenção global (GPT), mas como um sistema dinâmico que evolui no tempo.

Dada uma sequência de tokens $[x_1, \dots, x_T] \in \mathbb{R}^{T \times d}$, e SSM implementa f_θ por meio de transições de estado parametrizadas que podem ser espessas como uma recorrência de tempo contínua:

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t)$$

- $x(t)$: Entrada. O sinal ou a palavra que entra no sistema no instante t .
- $h(t)$: Estado oculto. É a “memória” do sistema. Guarda o que aconteceu no passado para influenciar o futuro.
- $h'(t)$: Taxa de mudança do estado.
- A, B, C : Matrizes de parâmetros que o modelo aprende durante o treinamento.
 - A controla como a memória é mantida ou esquecida
 - B controla como a nova entrada afeta essa memória.

Na prática, o sistema é discretizado para processar sequências de tokens. Computadores não trabalham bem com tempo contínuo, eles processam “tokens” (palavras/pixels) um por um.

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t$$

Aqui, em vez de uma derivada, temos uma **recorrência**. O estado atual h_t depende apenas do estado anterior $h_{\{t-1\}}$ e da entrada atual x_t .

As matrizes A e B descrevem o sistema no mundo real (contínuo). As versões $\bar{A}$ e $\bar{B}$ são a discretização delas para o mundo digital, onde cada passo no tempo tem uma duração Δ (step size).

- Δ - Step size. Define quando tempo do sistema contínuo cada token da sequência representa. Nas equações abaixo, é um escalar que multiplica a matriz.

$$\bar{A} = \exp(e^{\Delta A})$$

$$\bar{B} = [\exp(e^{\Delta A}) - I] A^{-1} B$$



De onde vêm essas fórmulas?

De onde vêm essas fórmulas? Bom, como temos uma Equação Diferencial, vamos achar A e B resolvendo ela.

$$h'(t) = Ah(t) + Bx(t)$$

Isso é uma **Equação Diferencial Linear Não-Homogênea**. A solução para essa equação é composta por duas partes.

- **A parte homogênea (Ah):** É como o sistema se comporta sozinho (a mola balançando). Como vimos, isso é resolvido por $e^{\{At\}}$.
- **A parte particular (Bx):** É como o sistema reage à estrada (os buracos, ou no seu caso, as palavras/tokens).

A solução geral no tempo contínuo é dada pela **fórmula de variação de parâmetros**:

$$h(t) = e^{\{At\}}h(0) + \int_0^t e^{\{A(t-\tau)\}} Bx(\tau) d\tau$$

Para transformar isso em código (discreto), assumimos que a entrada $x(t)$ é constante durante todo o pequeno intervalo Δ . Isso se chama Zero-Order Hold.

Seja $u = t - \tau$.

- Quando $\tau = 0 \implies u = t$.
- Quando $\tau = t \implies u = 0$.
- $\tau = -du$

$$h(t) = e^{At}h(0) - \int_t^0 e^{Au} Bx(\tau) du$$

$$h(t) = e^{At}h(0) - (e^{At}A^{-1}Bx(\tau))|_t^0$$

$$h(t) = e^{At}h(0) - (e^{A \cdot 0} - e^{At})A^{-1}Bx(\tau)$$

Uma exponencial matricial elevada a 0 é igual à matriz identidade.

$$h(t) = e^{At}h(0) + (e^{At} - I)A^{-1}Bx(t)$$

Tcharam! Chegamos em $| \{A\}$ e $| \{B\}$, que são os coeficientes que multiplicam $h(0)$ e $x(t)$.

Exponencial Matricial

Em contextos matemáticos, $\exp(x) = e^x$.

No artigo, $\exp()$ representa a função exponencial aplicada ao resultado da operação entre o escalar e a matriz A .

Como elevar e a uma matriz?

Não é possível elevar e a cada número da matriz. Para fazer isso, usamos Série de Taylor.

$$e^M = I + M + \frac{M^2}{2!} + \frac{M^3}{3!} + \dots + \frac{M^n}{n!}, \text{ onde } M^2 = M \times M$$

Calcular essa série finita é possível.

Por Diagonalização

Se $A = VDV^{\{-1\}}$ (é diagonalizável), então:

$$e^A = Ve^De^{\{-1\}}$$

Onde e^D é apenas uma matriz diagonal onde você eleva e a cada autovalor na diagonal principal.

$$e^D := \begin{pmatrix} e^{\lambda_1} & 0 & \dots \\ 0 & e^{\lambda_2} & \dots \\ \dots & \dots & \dots \end{pmatrix}$$

Demonstração:

Propriedade das matrizes diagonais:

$$D^k = \begin{pmatrix} \lambda_1^k & 0 & \dots \\ 0 & \lambda_2^k & \dots \\ \dots & \dots & \dots \end{pmatrix}$$

Substituindo $M = VDV^{\{-1\}}$ na Série de Taylor:

$$e^M = I + (VDV^{\{-1\}}) + \frac{(VDV^{\{-1\}})^2}{2!} + \dots$$

$$e^M = I + (VDV^{\{-1\}}) + \frac{VD^2V^{\{-1\}}}{2!} + \frac{VD^3V^{\{-1\}}}{3!} + \dots$$

$$e^M = V \left(I + D + \frac{D^2}{2!} + (D^3)(3!) + \dots \right) V^{-1}$$

O que sobrou dentro dos parênteses? Exatamente a série de Taylor de cada elemento na diagonal de D !

Mas isso ainda é uma série infinita... Bom, mas ela é conhecida. Seja matriz ou número:

$$e^M = \sum_0^{\infty} \frac{A^n}{n!}$$

$$I + D + \frac{D^2}{2!} + \dots = \begin{pmatrix} \sum_0^{\infty} \frac{\lambda_1^n}{n!} & 0 & \dots \\ 0 & \sum_0^{\infty} \frac{\lambda_2^n}{n!} & \dots \\ \dots & \dots & \dots \end{pmatrix} = \begin{pmatrix} e^{\lambda_1} & 0 & \dots \\ 0 & e^{\lambda_2} & \dots \\ \dots & \dots & \dots \end{pmatrix} := e^D$$

Acima, temos a definição de e^D , sendo D matriz diagonal. Por fim:

$$e^M = V e^D V^{-1}$$

Discretização Simplificada (Euler)

É barato computacionalmente, mas é menos preciso.

$$\bar{A} \approx I + \Delta A$$

3.3.2.5 Arquitetura Mamba (State-Space Modelling)

Introduz scanning seletivo onde $|\{A\}$ e $|\{B\}$ são modulados por porjeções lineares do token atual:

$$h_t = (g_t^A \odot \bar{A} h_{\{t-1\}}) + (g_t^B \odot \bar{B}) x_t, y_t = C h_t$$

$$g_t^A = \sigma(W_A x_t), g_t^B = \sigma(W_B x_t)$$

- W_A e W_B são matrizes de pesos aprendíveis.

Isso possibilita a dinâmica de transição a enfatizar inputs relevantes e suprimir os irrelevantes. Diferente do self-attention, que computa em paralelo similaridade entre os tokens, esse mecanismo possui escala linearmente em $\mathcal{O}(T)$.

A arquitetura Mamba-2 estende Mamba ao aprender $|\{A\}$ e $|\{B\}$ mdiratamente, sem a necessidade de mapeamento de contínuo para discreto com exponenciais duplas, dando estabilidade de treino para sequências longas.

3.3.2.6 Conclusão sobre Transformers e SSMs

Tanto Transformers quando SSMs implementam o mesmo mapeamento abstrato:

$$f_{\theta} : \mathcal{X} \rightarrow \mathcal{Z}, h_t = f_{\theta(X)}$$

- Transformers usam interações globais baseadas em conteúdo. Enfatizam agregação de contexto flexível.
- Mamba usa transições de estado aprendidas. Enfatiza modelamento dinâmico de tempo-contínuo.

3.3.3 Objetivos de Treinamento Self-Supervised

Modelos de fundação confiam em um ou mais tipos de Treinamento Auto-Supervisionado (SSL) para aprender representações de propósito geral de uma quantia enorme de dados não rotulados.

Em geral:

- $t(x)$ = Função objetivo do self-supervised training
- $\mathcal{L}$ = Loss, paradigma de treinamento em particular

Loss Function ($\mathcal{L}$)

Diz ao modelo o quanto ele está errando durante o treinamento. O objetivo do treinamento é minimizar esse valor.

Dado o resultado da Loss, o modelo usa esse valor para ajustar seus parâmetros internos (pesos) e tentar errar menos na próxima vez.

3.3.3.1 Masked Autoencoding

Máscara aplicada no input data ou no espaço latente. O modelo é treinado para reconstruir as partes escondidas a partir das partes visíveis. Assim, é forçado a aprender a estrutura e o contexto dos dados.

Abaixo temos a função do modelo. Ele recebe apenas a parte visível e tenta gerar uma previsão ($\hat{x}$) para o que foi escondido.

$$\hat{x} = f_{\theta}(x_{\setminus \Omega})$$

- x_{Ω} : São as partes do dado que foram escondidas (mascaradas). O conjunto Ω representa as posições ou índices dessas partes.
- $x_{\setminus \Omega}$: É o que restou, ou seja, as partes **visíveis** dos dados que o modelo pode ver.

Abaixo temos a função de perda (Loss).

$$\mathcal{L} = \frac{1}{|\Omega|} \sum_{i \in \Omega} || \hat{x}_i - x_i ||$$

O modelo é penalizado pela diferença entre o que ele previu (x^i) e o valor real que estava escondido (x_i). A soma é feita apenas sobre as posições mascaradas (Ω).

3.3.3.2 Autoregressive Models

O objetivo é tentar prever o próximo item de uma sequência. Usando a função Loss Cross-Entropy, o modelo é treinado para aprender a distribuição condicional. Comum em arquiteturas Tranformer decoder-based (GPT).

$$\mathcal{L}(\theta) = - \sum_{\{t=1\}}^T \log p_{\theta}(x_t | x_{\{<t\}})$$

- $x_{\{<t\}}$: É o contexto, ou seja, todos os tokens (palavras ou pedaços de palavras) que apareceram antes do momento atual t .
- x_t : É o “alvo” (target), o próximo token que o modelo deve acertar.
- $p_{\theta}(x_t|x_{\{<t\}})$: Esta é a probabilidade do próximo token ser x_t , dado que os anteriores foram $x_{\{<t\}}$. O modelo está aprendendo a distribuir probabilidades (ex: se o texto é “O gato subiu no...”, a probabilidade de “telhado” deve ser alta).
- **Logaritmo**: Se o modelo der uma probabilidade de 100% (1.0) para a palavra correta, $\log(1)=0$ (perda zero). Se ele der uma probabilidade muito baixa (perto de 0), o logaritmo será um número negativo muito grande.
- **Negativo**: Ele inverte o valor para que a “perda” seja um número positivo. O objetivo é minimizar esse número.
- **Soma**: O modelo soma os erros de cada previsão feita ao longo de toda a sequência (do tempo 1 até T)

3.3.3.3 Constrastive Learning Methods

Se nos exemplos anteriores o modelo tentava “reconstruir” ou “prever o próximo”, aqui o objetivo é **comparar**. A ideia é ensinar ao modelo o que são coisas parecidas e o que são coisas diferentes, sem labels.

Define o alvo $t(x)$ implicitamente através de relações de similaridade entre pares negativos (coisas que são diferentes) e positivos (coisas parecidas) no espaço latente $\mathcal{Z}$.

O framework SImCLR define pares positivos como duas versões da mesma imagem (como a foto original e a foto rotacionada). No CLIP, é uma imagem e o texto que a descreve.

O modelo projeta esses dados em um “espaço latente” $\mathcal{Z}$. O treinamento força os “pares positivos” a ficarem muito perto um do outro nesse espaço e os “pares negativos” a ficarem longe.

Abaixo temos a incorporação da amostra no espaço latente:

$$z_i = f_{\theta}(x_i)$$

A Loss InfoNCE é definida como:

$$\mathcal{L} = -\log \left(\frac{\exp\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right)}{\sum_{k=1}^N \frac{\exp(\text{sim}(z_i, z_k))}{\tau}} \right)$$

- **Numerador**: Mede a similaridade entre o par que **deveria** ser igual (z_i e z_j). Queremos que esse valor seja **alto**.
- **Denominador (Somatório)**: Soma a similaridade de z_i com **todos** os outros itens do lote (negativos). Queremos que essa soma seja **baixa** em relação ao numerador.
- **τ (Temperatura)**: Um parâmetro que controla o quão “rígida” é a comparação.
- **Logaritmo Negativo (-log)**: Assim como na entropia cruzada, transforma a probabilidade de acerto em um custo. Se a similaridade do par positivo for muito maior que a dos negativos, a perda será pequena.

3.3.4 Métodos de avaliação

Os modelos de fundação são avaliados por meio da análise de seu desempenho em diversas tarefas, como classificação, regressão, previsão e imputação.

Fine-tuning (Ajuste Fino)

Para avaliar o desempenho em tarefas discriminativas, uma camada de previsão leve é geralmente adicionada como camada final ao modelo. O modelo pode então sofrer Fine-Tuning, onde todos os pesos são treinados ainda mais com dados específicos da tarefa (como ECG), para estimar a performance em um cenário real.

Linear Probing (Sondagem Linear)

O modelo de fundação é congelado (os pesos pré-treinados não mudam) e adiciona-se apenas uma camada final bem simples (uma “cabeça” de classificação) e treina apenas essa camada. Isso dá insights da eficácia do pré-treino e a verdadeira capacidade do modelo de generalizar.

Teste de tarefas de previsão ou imputação

Para isso, zero-shot learning é aplicável, onde o modelo é usado diretamente em dados nunca antes vistos, sem módulo ou treinamento adicional. Métricas de avaliação comuns incluem acurácia, acurácia balanceada, AUPRC, AUROC, F1 e MSE.

3.4 EEG Foundation Models

3.4.1 Dados EEG e Features

Banco de dados mais usado e útil: Temple University Hospital Archive of Clinical Recordings

Para usar sinais EEG, a prática mais comum é dividir o sinal em pedaços que não se sobrepõem. Baseado no conhecimento do domínio do EEG, os sinais podem ser divididos em bandas de frequência, onde cada banda possui uma associação conhecida com diferentes estados do cérebro.

Outro passo comum é usar filtros highpass/lowpass para remover ruído.

Features extraídas manualmente podem ser passadas diretamente para o modelo ao invés do sinal. Alternativamente, features de frequência podem ser extraídas e combinadas com o input de séries temporais.

Embeddings espaciais informam ao modelo de qual eletrodo cada sample vem. Isso toma forma de um parâmetro treinável.

Embeddings temporais (comummente implementados como um parâmetro treinável) são adicionados após a tokenização para informar ao modelo a ordem temporal da sequência. Outra aproximação popular é aplicar sinusoidal positional encodings, como proposto no Transformer original.

3.4.2 Modelos de arquiteturas

Todos os EEG Foundation Models aqui consistem em esqueletos de Transformer ou Mamba, combinado módulos leves.

3.4.2.1 Encoding preliminar ou Convoluções

Convoluções sobre a dimensão temporal são operações matemáticas usadas para extrair características locais de um sinal bruto antes de enviá-lo para a rede principal.

Convolutional Neural Networks (CNNs)

São um tipo de arquitetura de Deep Learning projetada especificamente para processar dados que possuem uma estrutura de grade, como **imagens** (pixels) ou **áudio** (espectrogramas).

Diferente de uma rede neural comum, onde cada neurônio se conecta a todos os outros, a CNN foca em padrões locais

O funcionamento básico se divide em três etapas principais:

1. **Convolução (Filtros):** A rede desliza pequenos quadrados chamados “filtros” ou “kernels” sobre a imagem. Cada filtro é treinado para reconhecer uma característica específica, como uma borda vertical, uma curva ou uma cor.
2. **Pooling (Simplificação):** Após identificar as características, a rede reduz o tamanho da imagem para manter apenas as informações mais importantes. Isso torna o processamento mais rápido e ajuda a rede a ignorar variações irrelevantes (como o objeto estar um pouco mais para a esquerda ou direita).
3. **Camadas Totalmente Conectadas:** No final, a rede pega todos esses padrões detectados (olhos, orelhas, focinho) e decide se a imagem é, por exemplo, um “cachorro” ou um “gato”.

Antes das CNNs, para um computador entender uma imagem, era preciso descrever manualmente cada característica. .

Com as CNNs:

- **Aprendizado Automático:** A própria rede aprende quais características são importantes durante o treinamento.
- **Hierarquia de Padrões:** As primeiras camadas detectam coisas simples (linhas); as camadas do meio detectam formas (círculos); as últimas detectam objetos complexos (rostos).

3.4.2.2 Vector Quantization

Vector Quantization (VQ) é uma técnica de compressão e discretização de dados que transforma vetores contínuos (como sinais de áudio ou ondas cerebrais de um EEG) em uma série de símbolos discretos (tokens).

Imagine que você quer descrever cores infinitas usando apenas uma caixa de 12 lápis de cor. O VQ é o processo de olhar para uma cor específica e decidir qual dos 12 lápis é o mais parecido com ela.

—

O processo descrito no texto segue três etapas principais:

1. **Encoder (Codificador):** O dado bruto (como o sinal de EEG) é transformado em uma representação latente matemática, chamada de vetor h .
2. **Codebook (Dicionário):** Existe uma tabela (o **Codebook** V) que contém um número fixo de vetores padrão chamados “códigos” ou “centroides”. No texto, isso é definido como $V=\{v_1, \dots, v_K\}$.
3. **Busca do Vizinho Próximo:** O sistema compara o vetor de entrada h com todos os vetores do dicionário e escolhe aquele que tem a menor distância geométrica. A fórmula dada é:

$$z_i = \operatorname{argmin}_j || h_i - v_j ||$$

(Isso significa: “escolha o índice j do vetor v que está mais perto de h ”).

Existem variações disso:

- **Residual VQ (RVQ):** Em vez de usar apenas um dicionário, o modelo usa vários em série. Ele encontra o código mais próximo no primeiro dicionário, calcula o que sobrou (o erro ou “residual”) e tenta codificar esse erro no segundo dicionário. Isso permite uma precisão muito maior sem precisar de um dicionário gigante.
- **Múltiplos Codebooks:** Modelos como o **CodeBrain** usam dicionários separados para características diferentes, como tempo e frequência.

3.4.3 Métodos de avaliação

As tarefas mais comuns são classificação (estágio de sono, detecção de convulsão, reconhecimento de emoção) e regressão (estimação da atenção).

É mais comum fine-tunar um modelo para uma única tarefa, repetindo o processo para cada dataset de benchmark.

O processo de adaptação envolve adicionar uma cabeça simples de classificação consistindo em um FC Layer ou em um MLP leve.

FC Layer (Fully Connected Layer)

Tipo mais clássico de camada em redes neurais.

- **O que ela faz:** Cada neurônio desta camada está conectado a **todos** os neurônios da camada anterior.
- **Função:** Ela serve para combinar todas as características (features) aprendidas pelas camadas anteriores (como bordas, texturas ou padrões temporais) para chegar a uma conclusão global, como classificar uma imagem.
- **O Problema:** Como tudo está conectado a tudo, elas possuem muitos parâmetros, o que as torna “pesadas” e propensas a gastar muita memória e processamento.

Lightweight MLP (Multilayer Perceptron)

Um **MLP** é, essencialmente, uma rede composta por várias FC Layers empilhadas.

Um MLP leve usa estratégias para reduzir o custo computacional.

3.5 Modelos de fundação ECG

3.5.1 Arquitetura do modelo

A arquitetura de modelo de fundação mais popular consiste em um encoder CNN, seguido por um ou mais transformers.

3.5.2 Considerando relatórios textuais

Datasets de ECG de hospitais podem incluir relatórios escritos por profissionais, e pode ser interessante utilizar essa informação com os sinais correspondentes.

Wearable-Echo-FM pré treina uma LLM para codificar os relatórios, e aplica convoluções 1D para codificar o sinal ECG. Os encodings do ECG e do texto são alinhados com contrastive learning CLIP-style.

3.5.3 Objetivos de pré-treino

Modelos de fundação ECG são mais comumente treinados usando contrastive learning, incluindo a que usa CLIP para alinhar ECG e relatórios.

3.5.4 Métodos de avaliação

Todos os modelos de fundação ECG são avaliados com full fine-tuning e/ou linear probing (sondagem linear).

Linear Probing (Sondagem Linear)

o **Linear Probing** é uma técnica de avaliação para medir a qualidade das representações que o modelo aprendeu durante o pré-treinamento.

De forma simples: você “congela” o modelo principal e treina apenas uma camada final muito simples (linear) no topo dele para realizar uma tarefa específica.

—

3.5.4.1 Como funciona na prática

Imagine que você tem um modelo pré-treinado em bilhões de imagens da internet. Você quer saber se ele realmente “entende” o que é um tumor em um raio-X médico.

1. **Congelamento (Freezing):** Você não altera nenhum peso (parâmetro) do Foundation Model. Ele atua apenas como um extrator de características (**feature extractor**).
2. **Extração de Vetores:** Você passa suas imagens médicas pelo modelo. Ele gera um vetor de números (o **embedding**) que representa aquela imagem.
3. **Camada Linear:** Você adiciona uma única camada matemática simples ao final. Esta camada apenas multiplica os vetores por pesos para dar uma classificação (ex: “Saudável” ou “Doente”).
4. **Treino Mínimo:** Você treina **apenas** essa última camada. Se ela conseguir classificar bem os objetos, significa que o Foundation Model já tinha aprendido as características necessárias “sozinho” durante o pré-treinamento.

</aside>

3.6 Outros modelos de fundação unimodais de biosinais

3.6.1 Foundations Models para PPG

Para PPG, as features mais informativas ocorrem a frequências baixas (ritmos mais lentos). Os dados podem ser adquiridos em hospitais ou smartwatches (mas isso aumenta o low SNR - Singal-Noise Relation).

Para lidar com o ruído do mundo real, os autores não ignoraram os dados “sujos”. Eles treinaram o modelo diretamente neles para torná-lo **robusto**. Eles usam uma abordagem de **duas etapas**:

Etapa 1: Aprendendo os “Motifs” (Padrões)

O modelo busca pequenos padrões repetitivos no sinal, chamados **motifs**.

- **MAE (Masked Autoencoder):** O modelo tenta preencher partes omitidas do sinal para entender sua estrutura.
- **Convoluções Dilatadas e Cross-Attention:** Técnicas para o modelo “enxergar” o sinal em diferentes escalas de tempo e alinhar o que vê com um banco de padrões aprendidos.
- **Objetivo:** Criar uma **função de distância**. O modelo aprende a medir matematicamente o quão parecido um sinal é de outro com base nesses padrões.

Etapa 2: Refinamento com Aprendizado Contrastivo

Com a “métrica de semelhança” pronta, eles treinam um codificador **ResNet**.

- **Pares Positivos e Negativos:** O modelo aprende que sinais com motifs parecidos devem estar “perto” na sua memória interna, e sinais diferentes devem estar “longe”.
- **Contrastive Loss:** A função de erro ajusta o modelo para que a distância entre as representações neurais dos sinais seja igual à distância real observada nos sinais brutos.

3.6.1.1 Falha matemática no GPT-PPG

O modelo GPT-PPG usa transformers para prever PPG.

Os autores tentaram treinar o modelo de forma autorregressiva (prever o próximo pedaço do sinal com base nos anteriores) usando uma função de perda chamada **MSE (Mean Squared Error)**.

- **O que aconteceu:** O modelo “travou” em um mínimo local. Em vez de desenhar as ondas do coração com seus picos e vales, o modelo simplesmente previa uma linha reta (o valor médio do sinal).
- **Por que isso ocorre?** Sinais de PPG limpos e normalizados tendem a ser simétricos em relação à média. Para o MSE, se o modelo não tem certeza se o próximo ponto é um pico alto ou um vale baixo, a estratégia “matematicamente mais segura” para minimizar o erro é chutar o valor médio. É como se o modelo ficasse “em cima do muro”.

Para forçar o modelo a entender a incerteza e a forma real da onda, eles mudaram o objetivo do treinamento:

- **Previsão Probabilística:** Em vez de prever um único número (o valor do sinal), o modelo agora deve prever os **parâmetros de uma distribuição de probabilidade** (neste caso, a Logit-Laplace).
- **Vantagem:** Isso obriga o modelo a modelar a densidade do sinal. Ele não chuta apenas um ponto médio; ele entende a variação e as chances de o sinal estar em diferentes níveis, o que evita que ele colapse em uma previsão genérica e sem vida.

3.6.2 Foundation Models Para MEG

MEG usa magnetometers para medição. Campos magnéticos são pouco afetados pelo crânio, levando a um alto SNR. Assim, ganha do iEEG por ser não invasivo, mas o equipamento é caro.

Veremos duas arquiteturas: Wavenet e GPT-style transformer. Em ambos os casos:

Preparação dos Dados: A Tokenização

Diferente de palavras, sinais cerebrais são ondas contínuas. Para que o modelo os entenda como “tokens”, eles usam um processo de compressão:

- **Transformada μ -law:** É uma técnica usada em telecomunicações para comprimir a faixa dinâmica do sinal. Ela foca mais nos detalhes de sons (ou sinais) baixos e menos nos picos muito altos.
- **Quantização de 8-bits:** O sinal é transformado em 256 níveis discretos (como se você tivesse uma régua com 256 marcações). Isso permite que o modelo trate cada nível de voltagem como se fosse uma “palavra” em um dicionário.

3.6.3 Foundation Models de Biosinais Multimodais

3.6.3.1 Early Fusion

Em modelos multimodais (que usam áudio + texto, ou imagem + vídeo), a “fusão” é o momento em que os dados se misturam.

- **Early Fusion:** Os diferentes dados são combinados logo no início do processamento, transformando tudo em uma única representação antes de passarem pelas camadas principais do modelo.
- **O Desafio:** O texto aponta um problema crítico: a maioria dos dados de treino do BrainOmni vem de **uma única modalidade**. Ou seja, ele tem muitos dados que são **só** EEG e outros que são **só** MEG (não há alinhamento natural!)

Sobre o BrainOmni:

1. A Identidade dos Sensores (Sensor Embedding)

O modelo precisa saber de onde vem o dado para não misturar “alho com bugalhos”. Para isso, ele cria um **Sensor Embedding** combinando duas informações:

- **Type Embedding:** Informa se o sensor é de **EEG** ou **MEG** (e se é gradiente ou amplitude).
- **Spatial Embedding:** Informa a posição 3D exata do sensor na cabeça.

> Isso permite que o modelo trate canais diferentes de forma inteligente, mesmo que eles usem escalas de medida distintas. >

—

2. O Tokenizer (BrainOmni Tokenizer)

O objetivo aqui é transformar a onda contínua em símbolos discretos (tokens). O processo tem 4 passos:

1. **SEANet (Encoder)**: Usa convoluções para extrair características temporais (o “ritmo” do sinal).
2. **Cross-Attention**: Mistura a informação do tempo com a informação do sensor (os embeddings que vimos acima).
3. **RVQ (Residual Vector Quantization)**: Transforma essa mistura em códigos de um dicionário (como vimos na sua primeira imagem).
4. **SEANet (Decoder)**: Tenta reconstruir o sinal original a partir dos tokens para garantir que nenhuma informação vital foi perdida. O dicionário é atualizado usando **EMA** (Média Móvel Exponencial).

—

3. O “Cérebro” do Modelo (Criss-Cross Transformer)

Na segunda etapa, o modelo usa os tokens gerados para aprender de verdade:

- **Arquitetura**: Usa blocos de **Criss-Cross Transformer**. Esse nome sugere uma forma eficiente de olhar para as relações entre diferentes sensores e diferentes momentos no tempo simultaneamente.
- **Tarefa de Treino**: O modelo tenta prever “tokens mascarados” (escondidos). É como um jogo de completar lacunas no sinal cerebral.
- **Early Fusion (EMEG)**: Como prometido no texto anterior, aqui os sinais de EEG e MEG são tratados como uma única modalidade híbrida chamada “**EMEG**”. Eles passam pelos mesmos módulos de processamento, sem distinção de “muros” entre as tecnologias.

—

Resumo do Fluxo Total:

1. O sinal entra com sua etiqueta de **posição e tipo**.
2. O **Tokenizer** o “picota” em códigos (tokens) discretos.
3. O **Transformer** estuda esses códigos como se fossem frases de um texto, aprendendo a prever partes faltantes.

3.6.3.2 Deep Fusion

Enquanto na Early Fusion tudo é misturado logo na entrada, aqui o modelo mantém a identidade de cada sinal por mais tempo.

O modelo possui módulos dedicados para cada tipo de sinal (**modality-specific**), mas a integração real (a “fusão”) acontece nas camadas intermediárias do processamento.

O texto detalha um modelo específico que funciona assim:

- **Tokenização Separada:** Cada modalidade (EEG, MEG, etc.) passa por sua própria CNN 1D para gerar tokens.
- **Embeddings de Identidade:** São adicionados **temporal embeddings** (para saber quando o sinal ocorreu) e **modality embeddings** (para o modelo não esquecer qual sinal é qual).
- **Transformer Compartilhado:** Todos esses tokens, agora “etiquetados”, entram em um único Transformer. É aqui que a fusão profunda acontece.
- **Reconstrução Específica:** Na saída, cada modalidade tenta reconstruir seu sinal original a partir dessa representação compartilhada.

Os autores **removem aleatoriamente modalidades inteiras** durante o treino.

- **Por que?** Isso força o modelo a aprender a reconstruir, por exemplo, um sinal de EEG mesmo que ele só tenha recebido o sinal de MEG.
- **Resultado:** O modelo aprende as correlações profundas entre diferentes sinais biológicos.

PhysioOmni

Este é um modelo ainda mais complexo que usa **VQ Codebooks**:

- Ele tem dicionários (codebooks) específicos para cada modalidade e **um dicionário compartilhado** entre todas.
- **Alinhamento Temporal:** Ele usa as características do **EEG como âncora** para alinhar todos os outros sinais no tempo.
 - **Reconstrução:** Ele combina os códigos específicos e os compartilhados para prever o sinal final.

3.6.3.3 Late Fusion (Contrastive Learning)

Modelo os modelos de cada sinal são **totalmente independentes** e não compartilham componentes. Eles só se “conversam” no final do processo para alinhar o que aprenderam.

Nesta abordagem, você treina encoders separados para cada modalidade (ex: um para EEG, outro para batimento cardíaco). O objetivo não é misturar os sinais em um único processador, mas sim fazer com que as representações matemáticas finais de sinais que ocorreram ao mesmo tempo sejam **parecidas**.

Brant-X [94]: O Método CLIP para o Cérebro

O **Brant-X** usa uma lógica parecida com o modelo CLIP (que alinha imagens e textos):

- **Encoders Separados:** Ele usa um encoder pré-treinado para EEG e outro para sinais “EXG” (como movimentos oculares ou musculares).
- **Matriz de Similaridade:** Ele compara os “patches” (pedaços) de EEG com os de EXG. O modelo é treinado para que a diagonal dessa matriz (onde os sinais realmente coincidem no tempo) tenha a maior pontuação de similaridade possível.
- **Robustez:** Eles aumentam os dados (data augmentation) com diferentes taxas de amostragem para que o modelo não se perca se um sensor for mais rápido que o outro.

3.6.4 Métodos de Avaliação

Os modelos multimodais são avaliados de uma maneira similar aos modelos unimodais, de maneira simples:

- **Linear Probing:** Você congela o modelo e adiciona apenas uma “cabeça de predição linear” (uma camada matemática simples) no final. Se essa camada simples conseguir classificar bem os dados, o mérito é todo do modelo pré-treinado.
- **Fine-tuning:** Você permite que os pesos do modelo mudem um pouco para se ajustarem a uma tarefa específica (como diagnosticar uma arritmia).

3.7 Considerações de Design

Se referem a escolhas para influenciar a eficiência e a generalização.

3.7.1 Seleção de Estratégia de Tokenization

O desafio aqui é transformar uma onda contínua em pedaços (tokens) que o computador consiga processar, equilibrando **detalhe** e **velocidade**.

3.7.1.1 1. Tokenização Baseada em Pontos (Point-based)

É a abordagem mais “fiel” ao sinal original.

- **Como funciona:** Mantém quase todos os pontos de dados no tempo.
- **Vantagem:** Preserva as dinâmicas de alta frequência (detalhes muito rápidos e sutis).
- **Desvantagem:** É computacionalmente muito pesada, pois gera sequências gigantescas para o modelo processar.
- **Uso:** Análises onde a precisão temporal milimétrica é crítica.
- Em transformers: 1000 pontos $\rightarrow$ 1000x1000 comparações ($O(n^2)$)

3.7.1.2 2. Tokenização Baseada em Patches (Patch-based)

Esta é a **estratégia dominante** (usada em 32 de 50 modelos pesquisados).

- **Como funciona:** O sinal é dividido em segmentos de comprimento fixo (como “caixas” de tempo).
- **Vantagem:** Reduz drasticamente o custo computacional ao encurtar o tamanho da sequência, mantendo o contexto geral. (lembrando que os Transformers têm complexidade
- **Variações mencionadas:**
 - **Overlapping (Sobrepostos):** As caixas se sobrepõem, o que ajuda a reconstruir o sinal de forma mais suave, mas cria dados repetidos (redundância).
 - **Non-overlapping (Sem sobreposição):** É mais eficiente e rápido, mas pode perder precisão nas bordas onde um pedaço termina e o outro começa.
- Em transformers: 10 patches de 100 pontos (eram 1000 pontos) $\rightarrow$ 10x10 comparações (ao invés de comparar ponto com ponto, compara patche com patche)

3.7.1.3 3. Tokenização Guiada pelo Domínio (Motif-based)

Em vez de dividir o sinal de forma cega (por tempo ou pontos), esta técnica usa o **conhecimento médico/biológico**.

- **Como funciona:** O sinal é segmentado com base em eventos reais. Por exemplo, cada “pedaço” pode ser exatamente um batimento cardíaco completo em um ECG ou PPG.
- **Vantagem:** Preserva a estrutura fisiológica intrínseca. O modelo aprende sobre “batimentos” em vez de apenas “pedaços de ondas”. É o que vimos antes com os **motifs** no modelo Pulse-PPG.

3.7.2 Seleção de Arquitetura

3.7.2.1 1. Transformers vs. Mamba

O texto compara a arquitetura soberana atual com uma alternativa emergente:

- **Transformers:** São os favoritos pela alta **interpretabilidade** e pelo mecanismo de **self-attention** (que permite ao modelo focar em partes específicas do sinal). O problema é a **complexidade quadrática**: dobrar o tamanho do sinal quadruplica o custo de processamento, o que dificulta gravações longas.
- **State-Space Models (ex: Mamba):** Surgem como solução para dados contínuos e longos. Eles possuem **complexidade linear**, sendo muito mais rápidos e eficientes em termos de memória, mantendo uma capacidade de aprendizado competitiva.

> **Regra de ouro do texto:** Use **Mamba** se tiver pouco recurso computacional ou dados muito longos; use **Transformers** se precisar entender exatamente o porquê das decisões do modelo (são preferidos quando a interpretabilidade é a prioridade) >

—

3.7.2.2 2. Qual estilo de Transformer usar?

Nem todo Transformer é igual. A escolha depende do objetivo final:

- **Estilo BERT (Encoder):** Aprende representações bidirecionais (olha para o passado e futuro ao mesmo tempo). Excelente para **classificação** (ex: “esse sinal é de sono ou vigília?”).
- **Estilo GPT (Decoder):** Focado em prever o próximo ponto. Ideal para **geração de dados** e **previsão (forecasting)**.
- **Encoder-Decoder:** O melhor para tarefas de **tradução** ou transformação de sequência para sequência.

—

3.7.2.3 3. O Desafio do Espaço (Multi-canal)

Sinais como o EEG vêm de vários eletrodos espalhados pela cabeça. O modelo precisa entender a relação espacial entre eles:

- **Ordem Fixa:** Simples, mas o modelo só funciona com aquele layout específico de eletrodos.
- **Learnable Spatial Embeddings:** A abordagem mais comum. Permite que o modelo se adapte a diferentes posições de sensores.
- **Graph-based (Grafos):** A técnica mais avançada. Trata os sensores como pontos em um mapa (grafo) para entender a geometria real entre eles, oferecendo modelagem superior, mas com maior custo de complexidade.

3.7.3 Seleção de objetivo de pré-treino

Lembre-se que o objetivo do pré-treinamento serve para Self-Supervised Learning.

3.7.3.1 1. Alinhamento Contrastivo (estilo CLIP)

- **Quando usar:** Quando você tem **dados pareados** (ex: um sinal de EEG e um sinal de EMG gravados ao mesmo tempo, ou um sinal de ECG acompanhado de um relatório médico em texto).
 - **O objetivo:** O modelo aprende a aproximar as representações desses dois sinais diferentes na memória, entendendo que eles descrevem o mesmo evento biológico.
-

3.7.3.2 2. Objetivos MAE (Masked Autoencoder)

- **Quando usar:** Quando a prioridade é a **interpretabilidade da reconstrução**.
 - **O objetivo:** Você esconde (mascara) partes do sinal e pede ao modelo para “desenhar” o que falta.
 - **Vantagem:** Como o modelo precisa reconstruir o sinal original ou suas características, os pesquisadores conseguem ver visualmente se o modelo entendeu a forma da onda (picos, vales e ritmos).
-

3.7.3.3 3. Pré-treinamento Autorregressivo

- **Quando usar:** Quando o objetivo final (tarefa downstream) envolve **previsão (forecasting)**.
 - **O objetivo:** É o estilo “ChatGPT” aplicado a sinais. O modelo tenta prever o próximo ponto ou patch do sinal baseado nos anteriores.
 - **Vantagem:** É a melhor forma de treinar o modelo para entender a continuidade temporal e antecipar eventos futuros (como prever uma crise epilética antes dela ocorrer).
-

3.7.3.4 4. SimCLR (Contrastive Learning Flexível)

- **Quando usar:** É uma abordagem **mais genérica e flexível**.
- **O objetivo:** Diferente do CLIP (que usa dois sinais diferentes), o SimCLR geralmente pega o mesmo sinal, cria duas versões levemente modificadas dele (ex: adicionando um pouco de ruído) e treina o modelo para reconhecer que ambas vieram da mesma origem.
- **Vantagem:** Não precisa de dados pareados de sensores diferentes, sendo muito útil para grandes bancos de dados de uma única modalidade.

3.8 Desafios e direções futuras

3.8.1 Comparações justas e benchmarking

Comparar modelos é difícil, pois as tarefas específicas e as estratégias de adaptação variam de modelo pra modelo. Além disso, há divergência em quais datasets selecionar para avaliação.

Sets padrões de benchmark para EEG foram propostos,

- **Paradigma-específico: LibEER**, foca em datasets de reconhecimento de emoções. **SzCORE**, avalia detecção de convulsões.
- **ALMB**: Protocolo de avaliação de assuntos-independentes, onde o set de teste inclui apenas itens não vistos. Eles incluem cinco BCI Tasks com datasets específicos para prevenir correlações falsas.
- **AdaBrain-Bench**: Propõe uma seleção de 13 datasets EEG, avaliados sobre subject-independent e subject-dependent settings. Também inclui configuração de transferência de poucos disparos, onde o tanto de dados de fine-tuning é reduzido variando-se os graus.

Para prover algum insight sobre o desempenho dos foundation models de EEG, os modelos foram testados no **TUEG**, que é um dos maiores bancos de dados de EEG do mundo. Os autores focaram em dois subconjuntos específicos:

- **TUAB**: Usado para classificar se um EEG é **Normal** ou **Anormal**. É o teste básico de triagem médica.
- **TUEV**: Focado na classificação de eventos específicos (como diferentes tipos de crises ou padrões cerebrais).

O Physionet Computing in Cardiology Challenge 2021 é um benchmark estabelecido para modelos ECG. Contém grandes datasets de 12 gravações líderes da China, USA e Europa. Mais da metade dos modelos desse survey foram avaliados no PTB-XL, sendo o mais popular dataset de tarefas específicas para ECG.

Ainda existem: Chapman-Shaoxing, CPSC e CPSC Extra, Georgia, Ningbo, PTB and PTB-XL.

3.8.2 Explicabilidade e interpretabilidade

A interpretabilidade é um desafio, já que os modelos analisam sinais que são representações abstratas de processos biológicos. É importante fazer as decisões do modelo entendíveis, não apenas para pesquisadores, mas também para usuários finais, como clínicos, que precisam de confiança e transparência para aplicar esses sistemas em contexto hospitalar.

Explicações claras das previsões do modelo são necessárias para avaliar se o raciocínio foi plausível ou falso.

3.8.2.1 1. Estudos de Ablação (Ablation Studies)

É o método “científico clássico” de tentativa e erro.

- **Como funciona**: Os pesquisadores removem sistematicamente partes do modelo (uma camada, um tipo de dado ou um objetivo de treino) e observam o quanto a performance cai.
- **O objetivo**: Descobrir qual componente é realmente essencial e qual é apenas “enfeite”. Se você remove o sinal de MEG e a precisão não muda, significa que o modelo está ignorando o MEG.

3.8.2.2 2. Mapas de Atenção (Attention Maps)

Essenciais para modelos do tipo **Transformer**.

- **Como funciona**: Eles criam visualizações que mostram quais partes da sequência o modelo “olhou” com mais intensidade.

- **Utilidade:** Em modelos multimodais, eles revelam as conexões aprendidas entre sentidos diferentes (ex: o modelo focou no batimento cardíaco para entender uma mudança na onda cerebral).

3.8.2.3 3. Mapas de Saliência (Saliency Maps)

Diferente da atenção, que é interna do modelo, a saliência foca no **input**.

- **Como funciona:** Destaca as regiões temporais (momentos específicos) ou espaciais (quais eletrodos) que mais influenciaram a decisão final.
- **Exemplo:** Se o mapa de saliência brilhar intensamente em um pico de 3Hz no EEG durante um diagnóstico, o médico sabe que o modelo baseou sua decisão naquele padrão específico.

3.8.2.4 4. Testes Baseados em Perturbação (Perturbation-based Tests)

É uma forma de “estressar” o modelo para ver como ele reage.

- **Como funciona:** Os pesquisadores modificam o sinal de entrada de propósito. Exemplos citados:
 - **Jittering:** Adicionar tremores ou ruído.
 - **Frequency band occlusion:** Bloquear uma faixa de frequência (ex: esconder as ondas Alpha).
 - **Channel dropout:** Desligar alguns sensores.
- **O objetivo:** Verificar a robustez. Se o modelo errar tudo quando você desliga um único sensor, ele é frágil. Se ele continuar acertando, ele aprendeu características profundas e estáveis.

Agora veremos algo sobre o pré processamento.

—

3.8.2.5 1. O Perigo do Pré-processamento “Cego”

Muitos pesquisadores aplicam tratamentos padrão aos sinais antes de entregá-los à IA, mas isso pode deletar informações valiosas:

- **Filtros Passa-Banda (Bandpass Filtering):** Usados para remover ruídos (como batimentos cardíacos atrapalhando ondas cerebrais). O texto alerta que esses limites de frequência são escolhidos por convenção, e ninguém sabe exatamente quanta informação útil foi jogada fora junto com o ruído.
- **Reamostragem (Resampling):** Para facilitar o treino, sinais de diferentes aparelhos são colocados na mesma frequência. O risco aqui é o **downsampling**: reduzir a taxa de amostragem pode “apagar” dinâmicas rápidas e detalhes fisiológicos importantes que ocorrem em altas frequências.

—

3.8.2.6 2. A Solução: Multiescala e Multirresolução

Para evitar a perda de detalhes citada acima, o autor sugere que trabalhos futuros utilizem abordagens de **múltiplas escalas**. Em vez de forçar todo o sinal a ter uma única frequência

ou filtro, o modelo processaria o sinal em diferentes resoluções simultaneamente, preservando tanto o contexto geral quanto os micro-detahes.

3.8.2.7 3. Fidelidade de Reconstrução como Métrica (MAE)

Para modelos treinados com o objetivo de “completar a imagem” (Masked Autoencoders - MAE), o texto sugere uma nova forma de medir a qualidade:

- **Fidelidade de Reconstrução:** Não basta o modelo acertar a classificação; os pesquisadores devem mostrar o quão bem o modelo consegue reconstruir o sinal original.
- **Transparência:** Se um modelo reconstrói o sinal perfeitamente (alta fidelidade), mas falha na tarefa final, isso revela que a forma como ele “guarda” a informação na memória (representação latente) não é útil, apesar de ser precisa.

3.8.3 Pronto para aplicação no mundo real?

3.8.3.1 1. Eficiência e Recursos Limitados

O maior desafio é fazer modelos gigantes (como os Transformers) rodarem em tempo real em dispositivos pequenos (como smartwatches ou monitores portáteis).

- **Estratégias de compressão:** O texto cita técnicas para “enxugar” o modelo sem perder inteligência:
 - **Pruning (Poda):** Remover conexões neurais que não estão sendo usadas.
 - **Knowledge Distillation (Destilação de Conhecimento):** Treinar um modelo pequeno para imitar o comportamento de um modelo gigante.
- **Mudança de Arquitetura:** Reforça o uso de modelos **Mamba (SSM)**, que são mais rápidos por terem complexidade linear, ideais para sinais contínuos longos.

3.8.3.2 2. Privacidade e Ética (Federated Learning)

Dados médicos são extremamente sensíveis. Para treiná-los com segurança, o texto sugere o **Federated Learning (Aprendizado Federado)**.

- **Como funciona:** Em vez de os hospitais enviarem os dados brutos dos pacientes para uma central (o que seria perigoso), o modelo vai até o hospital, aprende com os dados locais e envia apenas o “conhecimento” aprendido de volta para a central. Os dados originais nunca saem da instituição de origem.

3.8.3.3 3. Regulação Médica

Para serem usados na prática clínica, esses modelos precisam passar por filtros rigorosos das autoridades de saúde, como a **FDA (EUA)** ou a **EU Medical Device Regulation (Europa)**. Isso garante que o software seja seguro, confiável e não dê diagnósticos errados por falhas técnicas.

3.8.3.4 4. O Desafio da Variabilidade (Generalização)

Cada corpo humano é único. O sinal de EEG de uma pessoa pode ser bem diferente do de outra devido à anatomia do crânio ou fisiologia.

- **Personalização:** O texto sugere um processo de **calibração leve**. O modelo geral é “ajustado” rapidamente para a fisiologia de um indivíduo específico sem precisar ser totalmente re-treinado.

Artigo 3: Foundation Models for Physiological Signals

4.1 Contexto e Modalidades de Sinais

A proliferação de sensores (smartwatches, anéis e adesivos) criou um ecossistema de dados em escala imensa, com previsões de mais de 1,1 bilhão de dispositivos em uso já em 2022. As principais modalidades exploradas são:

- **Fotopletismografia (PPG):** Técnica óptica que mede mudanças volumétricas na circulação sanguínea. Permite derivar métricas como frequência cardíaca, variabilidade da frequência cardíaca (VRC) e saturação de oxigênio (SpO_2).
- **Eletrocardiografia (ECG):** Padrão-ouro clínico para atividade elétrica cardíaca. A integração de ECG de canal único em dispositivos de consumo permite a detecção on-demand de arritmias, como a Fibrilação Atrial.
- **Unidades de Medida Inercial (IMUs):** Compostas por acelerômetros (ACC), giroscópios e magnetômetros. São fundamentais para rastrear atividade física, padrões de marcha e detectar quedas.
- **Outros Sinais:** Incluem Atividade Eletrodérmica (EDA/GSR) para medir estresse emocional e sensores de temperatura da pele.

4.2 Adaptação de Modelos de Fundação (TSFMs)

Os Modelos de Fundação para Séries Temporais (TSFMs) aprendem representações generalizáveis a partir de padrões complexos, reduzindo a dependência de engenharia de recursos manual.

- **Arquitetura:** O **Transformer** é a arquitetura dominante devido ao mecanismo de auto-atenção, que captura dependências temporais de longo alcance.
- **Tokenização e “Image-ification”:** Para lidar com sequências longas, os sinais são divididos em segmentos fixos chamados “patches”. Isso permite tratar o sinal como uma imagem 2D (tempo vs. canais), adaptando arquiteturas de sucesso como o **Vision Transformer** (ViT).
- **Multimodalidade (Hub-and-Spoke):** Modelos avançados processam fluxos de sensores individuais por codificadores específicos (“spokes”) e os fundem em um espaço de representação central compartilhado (“hub”).

4.3 Paradigmas de Pré-treinamento

A característica central dos MFs é a **Aprendizagem Supervisionada (SSL)**, que utiliza a estrutura dos dados para criar objetivos de aprendizagem sem rótulos humanos.

- **Modelagem de Sinal Mascarado:** O modelo reconstrói partes ocultas do sinal, aprendendo os padrões da fisiologia humana.
- **Aprendizagem Contrastiva:** Treina o modelo para aproximar amostras semanticamente semelhantes (ex: do mesmo indivíduo) e afastar amostras diferentes, capturando “assinaturas” fisiológicas únicas.
- **Estratégias Alternativas:**
 - **Destilação de Conhecimento:** Transfere conhecimento de sensores de alta fidelidade (ex: PPG) para sensores mais simples (ex: acelerômetro), criando sensores “virtuais”.

- **Alinhamento Sensor-Linguagem:** Alinha sinais de sensores com descrições em linguagem natural (ex: **SensorLM**).

4.4 Taxonomia dos Modelos Atuais

O survey categoriza os modelos em dois grandes grupos:

4.4.1 Modelos Multimodais de Larga Escala

- **LSM & LSM-2 (Google):** Pré-treinados em 40 milhões de horas de dados de 165 mil indivíduos. O LSM-2 introduziu a técnica **Adaptive and Inherited Masking** (AIM) para lidar com dados incompletos sem necessidade de imputação prévia.
- **NormWear:** Foca em configurações de sensores heterogêneas e capacidades **zero-shot** através do alinhamento com texto.

4.4.2 Modelos Focados em Domínio ou Aplicação

- **Apple PPG & ECG:** Um dos maiores esforços usando dados de dispositivos de consumo reais para aprender biomarcadores digitais.
- **SleepFM:** Modelo multimodal para dados de sono clínico, capaz de prever o início futuro de 130 doenças.
- **PAT (Pretrained Actigraphy Transformer):** Focado em dados de movimento para pesquisa em saúde mental e depressão.
- **LIFT-PD:** Especializado na detecção em tempo real de “congelamento de marcha” em pacientes com Parkinson.

4.5 Desafios e Futuro

Apesar do progresso, barreiras críticas impedem a tradução clínica generalizada:

- **Privacidade e Segurança:** Dados vestíveis são altamente sensíveis; técnicas como federation learning e differential privacy são exploradas, mas trazem desafios de performance.
- **Custo Computacional e Implementação:** O tamanho dos modelos dificulta a execução direta em smartwatches, exigindo processamento em nuvem (latência) ou técnicas de compressão (quantização e poda).
- **Viés Algorítmico:** Sensores PPG baseados em luz verde são menos precisos em tons de pele escuros devido à absorção de melanina, o que pode exacerbar disparidades raciais na saúde.
- **Validação Clínica:** Existe um abismo entre o sucesso em **benchmarks** e a utilidade clínica real, exigindo estudos prospectivos e ensaios clínicos randomizados.

Oportunidade Futura: A criação de uma “Ciência dos Dados Vestíveis” para definir taxas de amostragem e resoluções ideais que equilibrem a precisão do modelo com o consumo de bateria dos dispositivos.